

LockMeld: A Privacy-Preserving Cross-Chain Protocol for Confidential, Account-Based Blockchains

Hanqing Huang¹, Chenke Wang¹, Yu Long¹, Xian Xu², and Dawu Gu¹

¹ School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
{inivia, w_chenke, longyu, dwgu}@sjtu.edu.cn

² East China University of Science and Technology, Shanghai, China
xuxian@ecust.edu.cn

Abstract. In this paper, we present LockMeld, the first solution for enabling private cross-chain transfers when both underlying chains rely on homomorphic commitments to safeguard transaction amounts. LockMeld tackles the core challenges of ensuring unlinkability without sacrificing availability and accommodating arbitrary transaction amounts. Central to our solution is a batching technique that selectively discloses transaction details to the cross-chain intermediary, preventing any actor from directly correlating a sender’s escrow on one chain with the corresponding redemption on the other. Moreover, LockMeld combines additive homomorphic public-key encryption with randomizable signatures over randomizable commitments, ensuring robust on-chain confidentiality while still enabling necessary account management for future transactions. We provide not only a rigorous game-based security analysis but also demonstrate the protocol’s resilience against both malicious participants and external adversaries. We also implement and evaluate LockMeld’s performance. This empirical validation reveals that LockMeld’s privacy guarantees can be achieved in practice without incurring excessive overhead, making it an attractive option for privacy-conscious cross-chain interoperability.

Keywords: Account-based confidential blockchains · Privacy-preserving cross-chain · Atomic transfers

1 Introduction

Ensuring transaction privacy, specifically the confidentiality of amounts and the unlinkability of user identities, is a fundamental requirement for modern blockchain systems. While early solutions focused on UTXO-based models [12,16], recent research has shifted toward account-based confidential blockchains [6,3] due to their superior programmability and compatibility with the Ethereum ecosystem [13]. However, most existing protocols and their associated mixing strategies [7,19,4] are designed for single-ledger environments and operate in isolation.

Meanwhile, as cross-chain interoperability becomes essential for scaling liquidity and decentralized exchanges [1], guaranteeing privacy, particularly cross-ledger unlinkability, presents a critical challenge. Some existing single-chain privacy solutions may not naturally extend to these interactions, especially within the complex constraints of account-based confidential blockchains.

Technique Challenges. Designing privacy-preserving cross-chain protocols for account-based confidential blockchains introduces complexities beyond what is encountered in single-ledger solutions or nonconfidential-chain-focused cross-chain methods. We highlight two predominant challenges as follows.

- **Challenge 1: Unlinkability Without Sacrificing Availability.** In confidential settings, users must share transaction parameters (e.g., values and hiding factors) to process transfers. Sharing these details with a cross-chain intermediary exposes the user’s identity. Conversely, withholding this information prevents the intermediary from spending or transferring assets. Balancing unlinkability with transaction availability in a multi-ledger context remains non-trivial.
- **Challenge 2: Support for Arbitrary Transaction Amounts.** Existing privacy protocols often rely on fixed or pre-defined transaction denominations. In contrast, account-based confidential ledgers completely hide transaction amounts. Verifying cross-chain transfers of arbitrary amounts without undermining balance integrity is a key, yet complex, design requirement.

1.1 Solution Overview

- **Solution to Challenge 1:** We enhance existing coin-mixing methods by introducing a batching approach to address this problem. For clarity, we consider three pairs of honest senders and receivers engaged in a cross-chain payment. Senders and receivers maintain their accounts on different blockchains, namely Blockchain A and Blockchain B, respectively. Fig. 1 illustrates a naive adaptation of the coin-mixing method to a confidential chain. In this scenario, the Tumbler immediately links Receiver₁ and Sender₁ via the unique

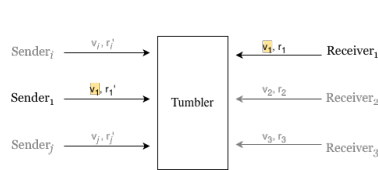


Fig. 1. The traditional coin-mixing method. v and r denote the transaction amount and the blinding factor, respectively. Yellow shading identifies information known to the tumbler that undermines unlinkability. $\{i, j\} = \{2, 3\}$.

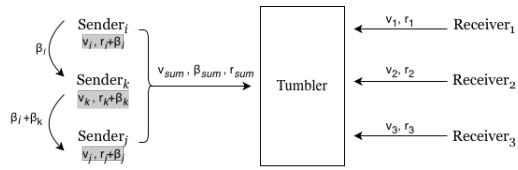


Fig. 2. The batching method. β denotes the randomness used to randomize r , while gray shading identifies information confidential to the tumbler. $\{i, j, k\} = \{1, 2, 3\}$.

transaction amount v_1 , as the Tumbler requires both the amount v and hiding factor r to spend the assets. To address this, we introduce a batching method that ensures cross-chain unlinkability while maintaining the Tumbler’s control over its account. As shown in Fig. 2, the Tumbler observes individual payment details and random factors on Blockchain B. However, on Blockchain A, the Tumbler only observes aggregated values ($v_{sum} = \sum v_i$ and $r_{sum} = \sum r_i + \beta_{sum}$) rather than individual parameters. By verifying the sum value v_{sum} , the Tumbler can validate the batch without being able to map specific Blockchain A transactions to their corresponding Blockchain B redemptions. This functionality relies on two core components: (1) *additively homomorphic public-key encryption*, which is used to protect the re-randomization factor β value and enables the aggregation of random factors without disclosing them individually; and (2) *randomizable signatures on randomizable commitments*, which facilitates commitments randomization while maintaining confidentiality. These tools allow the Tumbler to manage account balances across blockchains by providing sufficient cross-chain information while simultaneously preventing transaction linkage.

- **Solution to Challenge 2:** To support arbitrary transaction amounts, a key point is to ensure that the amount received on Blockchain B matches the corresponding value escrowed on Blockchain A. To this point, we integrate one-out-of-many proofs with the aforementioned techniques. This mechanism verifies the integrity of cross-chain transfers without disclosing individual values. Specifically, the proof confirms that a redemption on the target chain originates from a valid escrow on the source chain, while randomizable signatures on commitments maintain confidentiality. Continuing with Fig. 2 as an example, batching method confirms that $v_{sum} = v_1 + v_2 + v_3$. Then Sender_i can generate a one-out-of-many proof to prove that $v_i \in \{v_1, v_2, v_3\}$. Additionally, a randomizable signatures on commitment of v_i and $r_i + \beta_i$, ensures that it’s a legal commitment signed by Tumbler before and has not been tampered with. By enforcing these consistency checks, the protocol ensures both correctness and completeness for arbitrary-amount payments across distinct confidential ledgers.

1.2 Related Work

We briefly review two primary research strands and discuss their limitations in cross-chain contexts.

- *Mixing-based Protocols via One-out-of-Many Proofs:* Leading coin-mixing approaches for confidential blockchains [7,4,19,15] leverage specialized one-out-of-many proofs to ensure transaction unlinkability and confidentiality. However, these methods rely on the existence of a unified environment (e.g., a single smart contract or ledger) for transaction verification. Such a centralized verification model is not directly extensible to cross-chain settings, where state consistency are maintained across isolated and heterogeneous ledgers.

- *Anonymous Atomic Locks via Randomizable Puzzles*. This line of work, pioneered by A²L [18] and refined in A²L+ [9], evolved into BlindHub [14] to support arbitrary transaction amounts. While P2C2T [11] further extended these concepts to cross-chain transfers, it still leaves transaction details and amounts publicly visible. Although these protocols ensure atomicity and anonymity through puzzle randomization, they fail to satisfy the confidentiality requirements inherent in account-based private ledgers, where transaction values must remain hidden.

Table 1 compares our work with existing solutions across application scenarios, security properties, and privacy. While MPC or hardware-assisted schemes for non-confidential blockchains may be compatible with our protocol, they primarily address cross-chain atomicity rather than privacy. As these schemes generally omit privacy considerations, they are excluded from this comparison.

Table 1. Comparison of state-of-the-art privacy-preserving blockchain solutions.

	Scheme	Type	Model	Atomicity	Unlinkability	Confidentiality	Value
Intra-chain Private	A ² L [18], A ² L+ [9]	$\mathbb{P}$	U or A	●	●	○	Fixed
	BlindHub [14]	$\mathbb{P}$	U or A	●	●	●	Arbitrary
	MixCT [7], Veksel [4], BlindShuffler [19]	$\mathbb{C}^*$	Account	●	●	●	Arbitrary
	ValueShuffle [15]	$\mathbb{C}^*$	UTXO	●	●	●	Arbitrary
Inter-chain Private	P2C2T [11]	$\mathbb{P}$	U or A	●	●	○	Fixed
	zkCross [10]	$\mathbb{P}^*$	U or A	●	●	●	Fixed
	Our work	$\mathbb{C}$	Account	●	●	●	Arbitrary

Note: $\mathbb{P}$ and $\mathbb{C}$ denote public and confidential blockchains, respectively. Symbols U and A represent UTXO and Account-based models, respectively. A superscript * indicates a dependency on smart contracts; otherwise, the scheme relies solely on the basic functionality of the underlying chains. ● and ○ signify support and non-support, respectively.

1.3 Our Contributions

We propose the first private cross-chain protocol tailored for account-based confidential blockchains utilizing homomorphic commitments. Our specific contributions are as follows:

- **A new cross-chain solution: LockMeld.** We introduce LockMeld, the first cross-chain solution specifically designed for homomorphism-commitment-based confidential blockchains. LockMeld enables users on one blockchain to transfer arbitrary amounts of coins to another, ensuring both atomic execution and privacy throughout the process.
- **Rigorous Security Analysis.** We define the security and privacy properties of LockMeld within a formal game-based model. Our proofs demonstrate that the protocol ensures correctness and remains robust against a comprehensive range of adversarial behaviors.

- **Implementation and Evaluation.** We develop a prototype implementation of LockMeld, and substantiate its practicality through extensive empirical testing and performance evaluations.

2 Preliminaries

Basic Notations. For the security parameter $\lambda \in \mathbb{N}$, we denote the string of λ ones by 1^λ . When sampling an element r uniformly from a space $\mathcal{R}$, we use $r \leftarrow \$\mathcal{R}$. For $n \in \mathbb{N}$, $[n]$ refers to the set $\{1, 2, \dots, n\}$. We use the notation $y \leftarrow \phi(x; r)$ to represent a probabilistic polynomial time (PPT) algorithm ϕ , which upon receiving an input x and a random factor r , outputs y . Finally, $\text{negl}(\lambda)$ denotes a function that is negligible in the security parameter λ .

Let **GroupGen** be a PPT algorithm that on input 1^λ , outputs the description of a cyclic group $\mathbb{G}$ of prime order $p \geq 2^\lambda$ together with a random generator $g \leftarrow \$\mathbb{G}$. Then, $\mathbb{Z}_p = \{0, 1, \dots, p-1\}$ is the group of integers modulo p under addition, and is isomorphic to $\mathbb{G}$.

The underlying homomorphic message-hiding mechanisms, along with the cryptographic assumptions used later, are detailed in Appendix A.

2.1 One-Out-of-Many Proof

A one-out-of-many proof is a specific instance of a non-interactive zero-knowledge proof (NIZK), the formal definition of NIZK is provided in Appendix B.

One-out-of-many proof. Especially in our protocol, a prover needs to prove that in a set of n commitments $(c_1, c_2, \dots, c_n)$, there exists one committed with value 0 under public commitment key cmk . We can formally defined the relation with following equation:

$$\left\{ ((\text{cmk}, (c_1, c_2, \dots, c_n)), (l; r)) \left| \begin{array}{l} \forall i \in [n] : c_i \in C_{\text{cmk}} \\ \wedge l \in [n] \wedge r \in \mathcal{R} \\ \wedge c_l = \text{Com}_{\text{cmk}}(0; r) \end{array} \right. \right\} \quad (1)$$

The prover reveals neither the random mask r nor the index l of the target commitment in the commitment set. To achieve this, we define **OOOM** scheme with following algorithms.

- $\pi \leftarrow \text{OOOM.Prove}(\text{cmk}, (c_1, c_2, \dots, c_n), (l, r))$. On input the commitment key pk , commitment set $(c_1, c_2, \dots, c_n)$ and witness $(l; r)$, outputs the proof π .
- $0/1 \leftarrow \text{OOOM.Vf}(\text{cmk}, (c_1, c_2, \dots, c_n), \pi)$. Verifies the input commitment key cmk , commitment set $(c_1, c_2, \dots, c_n)$ and the proof π , outputs 0 if reject, and 1 if accept.

2.2 Adaptor Signature

An adaptor signature scheme can link the publishing of digital signature with the revealing of a specific secret value. We assume a hard relation R such that

$R(Y, y) = 1$ iff Y is a statement and y is the witness of it. The adaptor signature scheme consists of a signature scheme $(\text{KGen}, \text{Sig}, \text{Vf})$ and following algorithms.

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$. On input security parameter, output the public parameters pp used as implicit input of the following algorithms.
- $\tilde{\sigma} \leftarrow \text{PreSig}(\text{sk}, m, Y)$. On input the sign key sk , the message m and the statement Y , outputs a pre-signature $\tilde{\sigma}$ that can be adapted into a valid signature.
- $0/1 \leftarrow \text{PreVf}(\text{pk}, m, \tilde{\sigma}, Y)$. Verifies $\tilde{\sigma}$, outputs 0 if reject, or 1 otherwise.
- $\sigma \leftarrow \text{Adapt}(\tilde{\sigma}, y)$. On input y , the witness of statement Y , and the pre-signature $\tilde{\sigma}$, outputs the valid signature σ .
- $y' \leftarrow \text{Ext}(\sigma, \tilde{\sigma}, Y)$. Extract a secret value y' from the input. Where $y = y'$.

The adaptor signature scheme should provide pre-signature correctness, pre-signature adaptability, witness extractability and aEUF-CMA security. The formal description of these properties is shown in Appendix C.

2.3 Blinded Randomizable Signature

We use blinded randomizable signature scheme to enable randomizable signatures on a message m committed to c to realize unlinkability. The scheme consists of following algorithms.

- $\sigma^* \leftarrow \text{BlindSig}(\text{sk}, c)$. On input the sign key sk and the commitment c , outputs a blinded signature σ^* .
- $\sigma \leftarrow \text{UnblindSig}(\text{td}, \sigma^*)$. On input the decommitment information td and the blinded signature σ^* , outputs σ , a valid signature on message m .
- $\sigma' \leftarrow \text{RandSig}(\sigma)$. On input a valid signature σ , outputs a randomized signature σ' that remains valid.
- $1/0 \leftarrow \text{SigVerify}(\text{pk}, \sigma)$. Verifies σ under pk , and outputs 1 if it is valid.

2.4 Randomizable Puzzle

Following [18], the randomized puzzle scheme RandPuz is defined as follows.

- $(\text{pp}, \text{td}) \leftarrow \text{RandPuz.Setup}(1^\lambda)$. On input the security parameter 1^λ , outputs the public parameter pp and a trapdoor td .
- $(A, \alpha) \leftarrow \text{RandPuz.Gen}(\text{pp})$. On input the public parameter pp and a solution of puzzle α , outputs a randomizable puzzle A along with its solution α .
- $\alpha \leftarrow \text{RandPuz.Solve}(\text{td}, A)$. On input a trapdoor td and a puzzle A , outputs α , the solution of A .
- $(A', r) \leftarrow \text{RandPuz.PRand}(\text{pp}, A)$. On input the public parameter pp and a puzzle A with its solution α , outputs a randomized puzzle A' with its solution α' and a randomization factor r . Φ is a deterministic function satisfying $\alpha' = \Phi(\alpha, r)$.

A RandPuz scheme has the properties of correctness, security and privacy.

2.5 Randomizable Signatures on Randomizable Commitments

The RSoRC scheme consists of the following algorithms [14].

- $\text{pp} \leftarrow \text{RSoRC.Setup}(1^\lambda)$. On input a security parameter, outputs a public parameter pp .
- $(\hat{\text{sk}}, \hat{\text{pk}}) \leftarrow \text{RSoRC.Gen}(\text{pp})$. On input a public parameter pp , outputs a public key $\hat{\text{pk}}$ and a secret key $\hat{\text{sk}}$.
- $\hat{\sigma} \leftarrow \text{RSoRC.RCSig}(\hat{\text{sk}}, (c_1, c_2, \dots, c_n))$. On input a secret key $\hat{\text{sk}}$ and commitments $c_1, c_2, \dots, c_n$, outputs a signature $\hat{\sigma}$ on these commitments.
- $(c'_1, c'_2, \dots, c'_n, \hat{\sigma}') \leftarrow \text{RSoRC.RCRand}(c_1, c_2, \dots, c_n, \hat{\sigma}, r)$. On input commitments $c_1, c_2, \dots, c_n$, a corresponding signature $\hat{\sigma}$, and a randomization factor r , outputs randomized commitments and signature $c'_1, c'_2, \dots, c'_n$ and $\hat{\sigma}'$.
- $0/1 \leftarrow \text{RSoRC.RCVerify}(\hat{\text{pk}}, \hat{\sigma}', c'_1, c'_2, \dots, c'_n)$. On input a public key $\hat{\text{pk}}$, a signature $\hat{\sigma}'$ and commitments $c'_1, c'_2, \dots, c'_n$, outputs 0 or 1.

An RSoRC scheme should have the property of correctness and class-hiding.

3 The Confidential Cross-Chain Payment Problem

3.1 Underlying Chain Structure

Generally, our LockMeld system is constructed on top of blockchains that supports decentralized confidential transactions using homomorphic commitments.

- $\text{pp} \leftarrow \text{CT.Setup}(1^\lambda)$. Given a security parameter 1^λ , the *setup* algorithm outputs the public parameters pp . All subsequent algorithms implicitly assume the availability of these public parameters.
- $\text{ctx} \leftarrow \text{CT.Create}(\text{sk}_s, \text{pk}_r, c_v)^3$. In the *transaction creation* algorithm, without loss of generality, a confidential transaction ctx can be formally represented as $\text{ctx} = (\text{pk}_s, \text{pk}_r, c_v, \text{aux})$. Here, sk_s and pk_r denote the sender's secret key and the receiver's public key, respectively, while $c_v = \text{HCom.Com}(v; t)$ is the confidential (committed) form of the transaction amount v . aux is the application-dependent auxiliary information used to check the validity of the transaction. More specifically, $\text{aux} = (\pi_{\text{legal}}, \sigma)$. π_{legal} is used to validate the *integrity* (such as the range of v) of ctx , and the signature σ presents the *authority* from the sender.

For the sake of clarity, we also explicitly use $\text{ctx}^{\text{icp}} \leftarrow \text{CT.Create}(\text{sk}_s, \text{pk}_r, c_v)$ to denote an *incomplete* ctx which is well-formed except for the lacking of sk_s 's signature σ . Obviously, ctx^{icp} 's integrity can be publicly checked with π_{legal} .

- $1/0 \leftarrow \text{CT.Verify}(\text{ctx})$. Given a transaction ctx , the *transaction verification* algorithm outputs 1 if the transaction is valid and 0 otherwise. Because ctx is publicly available, any user can perform this verification.

³ We emphasize that in confidential chains built from homomorphic commitments, c_v alone can present the account states of the sender and the receiver.

Generally, we consider two confidential payment systems operating on separate blockchains that employ identical algorithms but distinct parameters and disjoint user sets. Crucially, an entity maintaining accounts on both chains is treated as two distinct participants within their respective user sets.

3.2 Entities

LockMeld involves three types of participants:

Senders. Senders are users who maintain accounts on Blockchain A with key pair (sk_s, pk_s) . They wish to send funds to accounts on Blockchain B, while preserving the confidentiality of both the transaction linkage and the transaction amounts from any third party.

Tumbler. The Tumbler serves as a trustless intermediary, enabling privacy-preserving cross-chain payments. It receives coins from senders on Blockchain A and sends coins to receivers on Blockchain B. In LockMeld, the Tumbler can be a third-party organization or entity acting as a cross-chain bridge, holding accounts and coins on both Blockchains A and B. The key pairs are denoted as (sk_{tA}, pk_{tA}) and (sk_{tB}, pk_{tB}) respectively. In addition, Tumbler holds HPKE key pair (sk_t^ψ, pk_t^ψ) (See Appendix A for more illustrations) and RSoRC key pair (sk_t, pk_t) due to functional requirements of the protocol. Besides, use (sk_t^*, pk_t^*) to denote the Tumbler's blind signature's signing key and verification key.

Receivers. Receivers hold accounts on Blockchain B with key pairs (sk_r, pk_r) and receive funds from senders via the Tumbler. To ensure privacy, each receiver maintains a confidential channel with the sender, typically established through a PKI-based key exchange.

3.3 Confidential Cross-Chain Payment System

A confidential cross-chain payment system is a tuple of PPT algorithms defined as below. Section 4.1 shows specific descriptions of this general construction.

- $pp \leftarrow \text{Setup}(1^\lambda)$. The public parameters pp are thereafter assumed to be implicit inputs during every phase of the protocol.
- $(oid_i, \sigma_i^*, td_i) \leftarrow \text{Register}(sk_t^*, pk_{r_i}, td_i)$. Here, oid_i denotes the identifier of pk_{s_i} 's lock transaction, and pk_{r_i} is the receiver's public key, which remains fixed. Then the Tumbler issues a blind signature σ_i^* on td_i and pk_{r_i} , without gaining knowledge of their exact contents, where td_i is the trapdoor for unblinding a blind signature and holding by the sender/receiver.
- $(oid_i, \sigma_r', v_r, t_r) \leftarrow \text{PuzzleRequest}(oid_i, \sigma_r^*, td_i)$. The **PuzzleRequest** algorithm is run by receivers. After registering, each receiver $pk_r \in \{pk_{r_i}\}_{i \in [n]}$ requests a confidential coin from the Tumbler in the form of an *incomplete* redeem transaction, by first proposing the following request. The receiver uses (oid, σ_r') to prove to the Tumbler that it matches a validly registered sender, where σ_r' is the unblinded result of σ_r^* with td_i . Besides, the receiver inform the Tumbler the (v_r, t_r) pair, where v_r is pk_r 's cross chain amount and t_r is the corresponding random factor used in the commitment.

- $(\text{ctx}_{\text{red}}^{\text{icp}}, A_r, \tilde{\sigma}_{tB}, \hat{\sigma}_r, \text{cpt}_{\alpha_r}, \text{state}, v_{\text{tot}}) \leftarrow \text{PuzzlePromise}(\text{oid}_i, \sigma'_r, v_r, t_r, \text{sk}_{tB}, \hat{\text{sk}}_t, \text{pk}_t^\psi, \text{pk}_t^*, \text{state})$. This algorithm is run by the Tumbler. It creates an incomplete confidential transaction for each $\text{pk}_r \in \{\text{pk}_{r_i}\}_{i \in [n]}$, where $\text{ctx}_{\text{red}}^{\text{icp}} = (\text{pk}_{tB}, \text{pk}_r, c_r, \pi_{\text{legal}})$, along with its presignature and other information.
- $(\text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i}, \beta_i, \text{cpt}_\beta) \leftarrow \text{PuzzleSolveRq}(c_{r_i}, A_i, \hat{\sigma}_{r_i}, \text{cpt}_{\alpha_i}, \text{sk}_{s_i}, \text{pk}_t^\psi, \text{cmk}, \text{cpt}_\beta)$. This algorithm is run by the sender with sk_{s_i} . The sender provides evidence that the escrow transaction ctx_{esc} matches the same amount as some “promised” redeem transaction without harming unlinkability.
- $1/0 \leftarrow \text{ProcessVerify}(\{(\text{pk}_{s_i}, \text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i})\}_{i \in [n]}, \{c_{r_i}, t_i\}_{i \in [n]}, v_{\text{tot}}, \text{cpt}_\beta, \text{pk}_t, \text{sk}_t^\psi, \text{state})$. The `ProcessVerify` algorithm is run by the Tumbler. The Tumbler runs this algorithm to decide whether to proceed to the next phase, i.e., by outputting 1 or 0. After verifying these tuples and the last β , the Tumbler proceeds to the next phase, or abort all requests otherwise.
- $\{\text{ctx}_{\text{esc}_i}\}_{i \in [n]}/0 \leftarrow \text{ProcessEscrow}(\{(\text{ctx}_{\text{esc}_i}^{\text{icp}}, \tilde{\sigma}_{s_i}, \text{cpt}'_{\alpha_i})\}_{i \in [n]}, \text{sk}_t^\psi)$. The `ProcessEscrow` algorithm is run by the Tumbler. The Tumbler decrypts each cpt'_{α_i} using sk_t^ψ to obtain the solution α'_i and then get the escrow transaction onchain. That is, the Tumbler can convert each $\tilde{\sigma}_{s_i}$ into a valid signature σ_{s_i} , yielding a finalized $\text{ctx}_{\text{esc}_i}$. Here we emphasize that instead of explicit transmission, α'_i is visible to pk_{s_i} once $\text{ctx}_{\text{esc}_i}$ is paid to the Tumbler, thanks to the extractability property of the underlying adaptor signature method.
- $\text{ctx}_{\text{red}_i} \leftarrow \text{ProcessRedeem}(\text{ctx}_{\text{red}_i}^{\text{icp}}, \tilde{\sigma}_{tB_i}, \beta_i, \sigma_{s_i}, \tilde{\sigma}_{s_i}, A'_i)$. The `ProcessVerify` algorithm is mainly run by the receiver, after getting the puzzle solution from the sender. Specifically, once the sender gets σ_{s_i} from the $\text{ctx}_{\text{esc}_i}$, it extracts α'_i from σ_{s_i} and pre-signature $\tilde{\sigma}_{s_i}$, then the sender can recover α_i using the random factor β_i , which can be used by the receiver to complete the Tumbler’s signature by forming the valid redeem transaction $\text{ctx}_{\text{red}_i}$.

3.4 Security Model

Previous works explored security guarantees for on-chain mixing services and cross-chain payments, yet none focused on privacy-preserving cross-chain protocols for confidential transactions. Before providing the formal security definitions, we list the security goals that LockMeld should achieve as follows.

- **Atomicity.** The sender’s payment to the Tumbler and the Tumbler’s payment to the receiver must either both succeed or both fail.
- **Unlinkability.** No entity, including the Tumbler, can link a redeem transaction to its corresponding escrow transaction, ensuring that only the sender and receiver maintain this linkage throughout and after the protocol.
- **Double-spent prevention.** A receiver cannot generate two successful redemption transactions from a single valid escrow transaction.
- **DoS resistance.** The protocol must resist Denial-of-Service (DoS) attacks. Every honest party should be able to finish the protocol without losing funds.
- **Confidential.** The coin’s value in the confidential transaction must remain unknown to all parties, including the Tumbler, except the sender and the

receiver. The satisfactory of this property means that the underlying confidentiality mechanism must remain robust.

– **Theft Prevention.** This goal has two aspects:

- At the system level, adversaries must not be able to redeem funds from the Tumbler without performing a valid escrow. That is, LockMeld could not be used to create redeeming coins without associated escrow.
- At the user level, adversaries must not be able to steal from honest users. A coin escrowed for a designated receiver cannot be redeemed by any other party (including the sender). That is, escrow transactions could only be redeemed by predetermined receivers.

– **Balance.** In each completed cross-chain payment, the amount in the escrow transaction equals the amount in the redeem transaction.

Oracles. To provide the formal definitions of these properties, we need to define the oracles that are accessible by the adversary. For length limitation, the detailed oracles could be found in Appendix D. Roughly, in the related security models, the challenger $\mathcal{C}$ maintains two tables, L_{pp} (the “puzzle promise” list) and L_{ps} (the “puzzle solver” list). Table 4 summarizes the structure of these lists. For a table L , we let $L[x]$ to denote the row where the value of the value of X is x . We use $L.Y$ to denote the column Y of L , and write $L[x].Y$ to refer to the value in column Y of the row corresponding to x (i.e., a specific cell).

Below, we present the formal security properties of a LockMeld scheme.⁴

Definition 1 (Atomicity) *The LockMeld scheme achieves atomicity if, for any PPT adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{atomicity}}(\lambda) = \Pr[\text{Atomicity}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \leq \text{negl}(\lambda)$. The experiment $\text{Atomicity}_{\text{LockMeld}}^{\mathcal{A}}$ is defined as Fig 3.*

Definition 2 (Unlinkability) *The LockMeld scheme achieves unlinkability if, for any PPT adversary $\mathcal{A}$,*

$\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{unlink}}(\lambda) = \left| \Pr[\text{Unlinkability}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$. *The experiment $\text{Unlinkability}_{\text{LockMeld}}^{\mathcal{A}}$ is defined as Fig 5.*

Intuitively, the adversary cannot distinguish whether a puzzle solving request matches a particular puzzle promise if both keys remain uncorrupted.

Definition 3 (Double-spent prevention) *The LockMeld scheme can prevent double spending if, for any PPT adversary $\mathcal{A}$,*

$\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{doublespend}}(\lambda) = \Pr[\text{DSPrevention}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \leq \text{negl}(\lambda)$. *The experiment $\text{DSPrevention}_{\text{LockMeld}}^{\mathcal{A}}$ is defined as Fig 4.*

This property ensures that the same escrowed coin cannot be redeemed twice.

⁴ In the preceding subsection, we defined **ProcessEscrow** and **ProcessRedeem** with the output of `ctxs` or 0. For simplicity in the subsequent explanation of our security games, we replace `ctx` with 1 to indicate successful generation and still use 0 to indicate failure.

Definition 4 (DoS Resistance) *The LockMeld scheme is DoS-resistant if an adversary cannot indefinitely prevent honest protocol participation.*

Definition 5 (Confidentiality) *The LockMeld scheme achieves confidentiality if, for any PPT adversary $\mathcal{A}$,*

$$\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{confidentiality}}(\lambda) = \left| \Pr[\text{Confidentiality}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda),$$

where the experiment $\text{Confidentiality}_{\text{LockMeld}}^{\mathcal{A}}$ is defined as Fig 6.

Definition 6 (Theft Prevention) *The LockMeld scheme prevents theft if, for any PPT adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{theftprev}}(\lambda) = \Pr[\text{TheftPrevention}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \leq \text{negl}(\lambda)$, where $\text{TheftPrevention}_{\text{LockMeld}}^{\mathcal{A}}$ is defined as Fig 7.*

This definition ensures that no adversary can redeem someone else's escrowed coin or create a redemption out of nothing.

Definition 7 (Balance) *The LockMeld scheme is balanced, if for any PPT adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{balance}}(\lambda) = \Pr[\text{Balance}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \leq \text{negl}(\lambda)$, where $\text{Balance}_{\text{LockMeld}}^{\mathcal{A}}$ is defined as Fig 8.*

The scheme is balanced if an adversary cannot redeem a puzzle for either more or less than the originally escrowed amount.

Atomicity $_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ 
2:  $(\text{pk}_r, \text{pk}_s, \text{ct}_{\text{red}, \mathcal{A}}^{\text{icp}}, \text{ct}_{\text{esc}, \mathcal{A}}^{\text{icp}}, \text{aux}_{\mathcal{A}}, \mathbb{L}) \leftarrow \mathcal{A}^{\text{OCT}}(\text{pp})$ 
   // *aux is shorthand for the parameters related to  $\mathcal{A}$ . */
3:  $b_0 \leftarrow \text{ProcessEscrow}(\text{ct}_{\text{esc}, \mathcal{A}}^{\text{icp}}, \tilde{\sigma}_{s, \mathcal{A}}, \text{cpl}_{\alpha, \mathcal{A}}', \text{sk}_t^\psi)$ 
4:  $b_1 \leftarrow \text{ProcessRedeem}(\text{ct}_{\text{red}, \mathcal{A}}^{\text{icp}}, \tilde{\sigma}_{tB, \mathcal{A}}, \beta_{\mathcal{A}}, \sigma_{s, \mathcal{A}}, \tilde{\sigma}_{s, \mathcal{A}}, A'_{\mathcal{A}})$ 
5: return  $b_0 \oplus b_1$ 
```

Fig. 3. Definition of Atomicity.

DSPrevention $_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ 
2:  $(\text{ct}_{\text{red}, \mathcal{A}}^{\text{icp}}, \tilde{\sigma}_{tB, \mathcal{A}}, \beta_{\mathcal{A}}, \sigma_{s, \mathcal{A}}, \tilde{\sigma}_{s, \mathcal{A}}, A'_{\mathcal{A}}) \leftarrow \mathcal{A}^{\text{OCT}}(\text{pp})$ 
3:  $b \leftarrow \text{ProcessRedeem}(\text{ct}_{\text{red}, \mathcal{A}}^{\text{icp}}, \tilde{\sigma}_{tB, \mathcal{A}}, \beta_{\mathcal{A}}, \sigma_{s, \mathcal{A}}, \tilde{\sigma}_{s, \mathcal{A}}, A'_{\mathcal{A}})$ 
4:  $b' := (\exists A_{\text{dsp}}, L_{\text{ps}}[A_{\text{dsp}}].A = L_{\text{ps}}[A'_{\mathcal{A}}].A)$ 
   // *The same escrowed coin is redeemed twice. */
5: return  $b \wedge b'$ 
```

Fig. 4. Definition of Double-spent prevention.

Unlinkability $_{UMCT}^{\mathcal{A}}(1^\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ 
2:  $\{(\text{ct}_{\text{red}}^{\text{icp}(i)}, c_r^{(i)}, A^{(i)}, \tilde{\sigma}_r^{(i)}, \text{cpl}_{\alpha}^{(i)})\}_{i \in \{0,1\}} \leftarrow \mathcal{A}^{\text{Onc}}(\text{pp})$ 
3:  $b \leftarrow \{0, 1\}$ 
4:  $(\text{ct}_{\text{esc}}^{\text{icp}(b)}, A^{(b)}, \tilde{\sigma}_s^{(b)}, \text{cpl}_{\alpha}^{(b)}, \pi_{\text{valid}}^{(b)}, \beta^{(b)}, \text{cpl}_{\beta}) \leftarrow$ 
    $\text{PuzzleSolveRq}(c_r^{(b)}, A^{(b)}, \tilde{\sigma}_r^{(b)}, \text{cpl}_{\alpha}^{(b)}, \text{sk}_t^\psi, \text{cmk}, \text{cpl}_{\beta})$ 
5:  $1 \leftarrow \text{ProcessVerify}(\{(\text{pk}_{s_i}, \text{ct}_{\text{esc}, i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \text{cpl}_{\alpha_i}', \pi_{\text{valid}_i})\}_{i \in [n]},$ 
    $\{c_{r_i}, t_i\}_{i \in [n]}, v_{\text{tot}}, \text{cpl}_{\beta}, \text{pk}_t, \text{sk}_t^\psi, \text{state})$ 
6:  $1 \leftarrow \text{ProcessEscrow}(\{(\text{ct}_{\text{esc}, i}^{\text{icp}}, \tilde{\sigma}_{s_i}, \text{cpl}_{\alpha_i}')\}_{i \in [n]}, \text{sk}_t^\psi)$ 
7:  $1 \leftarrow \text{ProcessRedeem}(\text{ct}_{\text{red}}^{\text{icp}(b)}, \tilde{\sigma}_{tB}^{(b)}, \beta^{(b)}, \sigma_s^{(b)}, \tilde{\sigma}_s^{(b)}, A^{(b)})$ 
8:  $b' \leftarrow \mathcal{A}^{\text{OCT}}(\text{ct}_{\text{esc}}^{\text{icp}(b)}, A^{(b)}, \tilde{\sigma}_s^{(b)}, \sigma_s^{(b)}, \text{cpl}_{\alpha}^{(b)}, \pi_{\text{valid}}^{(b)})$ 
9:  $b_0 := (L_{\text{pp}}[\text{pk}_r].\text{Corrupt} = 0)$ 
10:  $b_1 := (L_{\text{ps}}[\text{pk}_s].\text{Corrupt} = 0)$ 
   // *Public keys involved are uncorrupted. */
11: return  $b_0 \wedge b_1 \wedge (b = b')$ 
```

Fig. 5. Definition of Unlinkability.

Confidentiality $_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ 
2:  $(\text{pk}_r, \text{pk}_s, v_r^{(0)}, v_r^{(1)}) \leftarrow \mathcal{A}^{\text{OCT}}(\text{pp})$ 
3:  $b \leftarrow \{0, 1\}$ 
   // *Parameters unrelated to b are omitted in the following algorithms. */
4:  $(v_r^{(b)}) \leftarrow \text{PuzzleRequest}(v_r^{(b)})$ 
5:  $(\text{ct}_{\text{red}}^{\text{icp}(b)}, \tilde{\sigma}_r^{(b)}, \text{state}^{(b)}, v_{\text{tot}}^{(b)}) \leftarrow \text{PuzzlePromise}(v_r^{(b)}, \text{state})$ 
6:  $(\text{ct}_{\text{esc}}^{\text{icp}(b)}, \tilde{\sigma}_s^{(b)}, \pi_{\text{valid}_i}^{(b)}) \leftarrow \text{PuzzleSolveRq}(c_r^{(b)}, \tilde{\sigma}_r^{(b)})$ 
7:  $b_0 \leftarrow \text{ProcessVerify}(\{(\text{ct}_{\text{esc}, i}^{\text{icp}}, \tilde{\sigma}_{s_i}', \pi_{\text{valid}_i})\}_{i \in [n]}, \{c_{r_i}, t_i\}_{i \in [n]}, v_{\text{tot}}^{(b)}, \text{state}^{(b)})$ 
8:  $1 \leftarrow \text{ProcessEscrow}(\{(\text{ct}_{\text{esc}, i}^{\text{icp}}, \tilde{\sigma}_{s_i}', \pi_{\text{valid}_i})\}_{i \in [n]})$ 
9:  $1 \leftarrow \text{ProcessRedeem}(\text{ct}_{\text{red}}^{\text{icp}(b)})$ 
10:  $b' \leftarrow \mathcal{A}^{\text{Onc}}(\text{ct}_{\text{red}}^{\text{icp}(b)}, \text{ct}_{\text{esc}}^{\text{icp}(b)}, \tilde{\sigma}_r^{(b)}, \tilde{\sigma}_s^{(b)}, \text{state}^{(b)}, v_{\text{tot}}^{(b)})$ 
11:  $b_1 := (L_{\text{pp}}[\text{pk}_r].\text{Corrupt} = 0) \wedge (L_{\text{ps}}[\text{pk}_s].\text{Corrupt} = 0)$ 
   // *Public keys involved are uncorrupted. */
12: return  $b_0 \wedge b_1 \wedge (b = b')$ 
```

Fig. 6. Definition of Confidentiality.

```

TheftPreventionALockMeld(1λ)
1: pp ← Setup(1λ)
2: (ctxicpred,A, σtB,A, βA, σs,A, σ̃s,A, A'A) ← AOCT(pp)
3: b ← ProcessRedeem(ctxicpred,A, σtB,A, βA, σs,A, σ̃s,A, A'A)
4: b0 := (Lps[A'A].pkr ≠ pkA) ∧ (Lpp[(Lps[A'A].pkr)).Corrupt = 0)
   // * Redeem from an honest user */
5: b1 := (not ∃pks s.t. Lps[pks].pkr = pkA)
   // * Redeem from nothing */
6: return b ∧ (b0 ∨ b1)

```

Fig. 7. Definition of Theft prevention.

```

BalanceALockMeld(1λ)
1: pp ← Setup(1λ)
2: (ctxicpred,A, ctxescA, σtB,A, σ̃s,A, βA, A'A, AA, cplα,A, πvalid,A) ← AOCT(pp)
3: b0 ← ProcessVerify({(pksi, ctxicpesc,i, A'i, σ̃si, cplαi, πvalidi)}i∈[n],
   {cri, li}i∈[n], vtot, cplβ, pkt, sktψ, state)
4: b1 ← ProcessEscrow({(ctxicpesc,i, σ̃si, cplαi)}i∈[n], sktψ)
5: b2 ← ProcessRedeem(ctxicpred,A, σtB,A, βA, σs,A, σ̃s,A, A'A)
6: b3 := (Lps[A'A].vs = Lpp[AA].vr) ∧ (Lps[A'A].A ≠ AA)
   // *Same escrow & redemption value using different puzzles. */
7: return b0 ∧ b1 ∧ b2 ∧ b3

```

Fig. 8. Definition of Balance.

3.5 Further Robustness Analysis

While we formally define an adversary's ability to actively attack and profit from the protocol and provide how our cryptographic design mitigates these threats in the later section, we assume certain passive behaviors (e.g., being offline or sending malformed messages) do not occur, to ensure the liveness. These behaviors and their associated consequences are summarized in Table 2.

Table 2. Impact of Node Abortions and Passive Misbehaviors

Case	Entity	Behavior	Abortion	Security
1	S_i	offline	timeout	Y
2	T	offline	timeout	Y
3	R_i	offline before ProcessEscrow	timeout	Y
4	R_i	offline after ProcessEscrow	timeout	N
5	T	PuzzlePromise	ProcessEscrow	Y
6	S_i	PuzzleSolveRq	ProcessVerify	Y

Note: Offline indicates that the entity is incapable of sending messages or performing on-chain operations. Timeout refers to the refunding of locked coins. Y and N signify that atomicity (security) is preserved or compromised, respectively.

4 Protocol of LockMeld

4.1 General Construction

The LockMeld scheme operates in discrete epochs, each consisting of four phases: Register, Puzzle Promise, Escrow, and Puzzle Solving. Fig. 9 provides a high-level overview of the algorithm.

Within a single epoch, the Tumbler functions not only as a cross-chain bridge but also as a protector of privacy by obscuring the linkage between senders and receivers. Additionally, transactions' amounts remain hidden from everyone

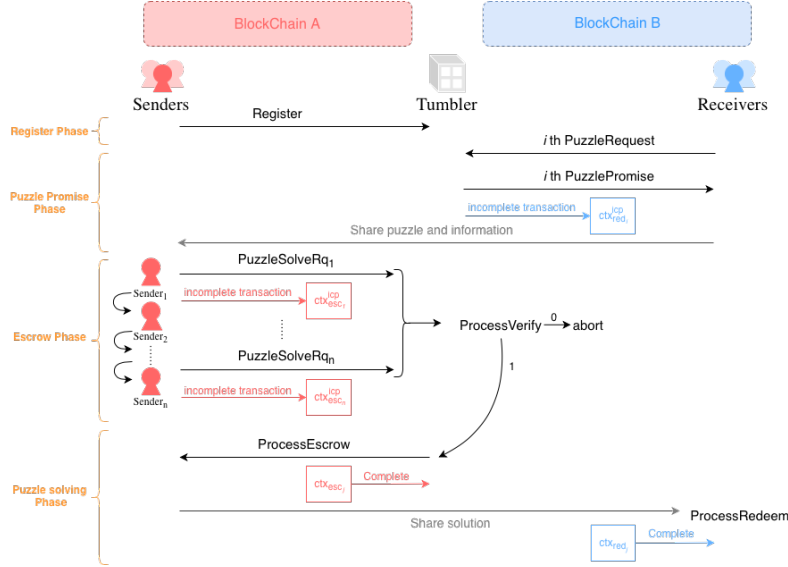


Fig. 9. Overview of the LockMeld architecture for n sender-receiver pairs. The escrow phase follows a sequential batching process, whereas subsequent phases are executed independently for each $i, j \in [n]$. Red (or blue) squares represent transactions on BlockChain A (or B), respectively. Solid black arrows denote interactions involving the Tumbler, while gray arrows indicate private peer-to-peer (P2P) communication.

except the sender and the receiver, including the Tumbler itself. The details of this construction are outlined below. We emphasize that there is a confidential channel between each sender-receiver pair and thus ignore the secret information transformation process within each pair.

Setup phase. LockMeld begins with a global setup phase, which produces all the publicly available parameters to be used in subsequent algorithms:

- $pp \leftarrow \text{Setup}(1^\lambda)$. The public parameters pp are thereafter assumed to be implicit inputs during every phase of the protocol.

Register phase. To counteract Denial-of-Service (DoS) attacks, each sender is required to lock a specific amount of coins (e.g., the upper limit of the mixing amount during this epoch) for the duration of the service. In this phase, the sender provides the identifier of the lock transaction, and the receiver proves the existence of that lock transaction, which originates from the sender and is bound to the receiver, without revealing its exact linkage. Suppose there are n senders in total, denoted as pk_{s_i} ($i \in [n]$). The registration process for each sender can be summarized as follows.

- $(oid_i, \sigma_i^*, td_i) \leftarrow \text{Register}(sk_t^*, pk_{r_i}, td_i)$. Here, oid_i denotes the identifier of pk_{s_i} 's lock transaction, and pk_{r_i} is the receiver's public key, which remains fixed. Then the Tumbler issues a blind signature σ_i^* on td_i and pk_{r_i} , with-

out gaining knowledge of their exact contents, where td_i is the trapdoor for unblinding a blind signature and holding by the sender/receiver.

Puzzle promise phase. During this phase, the Tumbler provides each of the n receivers with a *conditional* redeem transaction, via interaction. The n redeem transactions become valid if and only if each of the n receiver can subsequently present a correct solution to a carefully constructed puzzle in the latter phase. In this phase, n receivers conduct the following steps within a pre-defined time window, i.e., before the puzzle promise phase ends.

- $(oid_i, \sigma'_r, v_r, t_r) \leftarrow \text{PuzzleRequest}(oid_i, \sigma_r^*, td_i)$. The **PuzzleRequest** algorithm is run by receivers. After registering, each receiver $pk_r \in \{pk_{r_i}\}_{i \in [n]}$ requests a confidential coin from the Tumbler in the form of an *incomplete* redeem transaction, by first proposing the following request. The receiver uses (oid, σ'_r) to prove to the Tumbler that it matches a validly registered receiver, where σ'_r is the unblinded result of σ_r^* with td_i . Besides, the receiver inform the Tumbler the (v_r, t_r) pair, where v_r is pk_r 's cross chain amount and t_r is the corresponding random factor used in the commitment.
- $(ctx_{red}^{icp}, A_r, \tilde{\sigma}_{tB}, \hat{\sigma}_r, cpt_{\alpha_r}, \text{state}, v_{tot}) \leftarrow \text{PuzzlePromise}(oid_i, \sigma'_r, v_r, t_r, sk_{tB}, \hat{sk}_t, pk_t^\psi, pk_t^*, \text{state})$. The **PuzzlePromise** algorithm is run by the Tumbler. Tumbler creates an incomplete confidential transaction for each $pk_r \in \{pk_{r_i}\}_{i \in [n]}$, where $ctx_{red}^{icp} = (pk_{tB}, pk_r, c_r, \pi_{legal})$, along with its presignature and other information, as follows.
 1. Check the validity of oid_i, σ'_r , and the rest balance is enough for v_r using the function of underlying chain.
 2. Compute ctx_{red}^{icp} , which is a transaction that the Tumbler sends a confidential coin $c_r = \text{HCom.Com}(v_r; t_r)$ to the the receiver with pk_r .
 3. Randomly generate a hard puzzle A_r along with its solution α_r .
 4. Encrypt the solution α_r into cpt_{α_r} via the Tumbler's HPKE encryption key pk_t^ψ , thereby creating a randomizable puzzle solution in ciphertext form that can be recovered in following phases.
 5. Presign the incomplete redeem transaction ctx_{red}^{icp} with the puzzle A_r and the Tumbler's secret key sk_{tB} , yielding the presignature $\tilde{\sigma}_{tB}$.
 6. Bind the puzzle A_r and the committed coin c_r together within ctx_{red} using $\hat{\sigma}_r$ via RSoRC to prevent receiver and sender from colluding and making Tumbler solving a different puzzle, which leads to unbalance as it can be used in a redemption of higher transaction value.
 7. Update the internal state $\text{state} \leftarrow \text{state} \cup \{c_r\}$. state is initially empty.
 8. At last, the Tumbler sends $(ctx_{red}^{icp}, A_r, \tilde{\sigma}_{tB}, \hat{\sigma}_r, cpt_{\alpha_r})$ to the receiver with pk_r . Note the range proof of c_r and balance guarantee are ensured by the underline blockchain in the π_{legal} part.

Upon completion of this phase for all n receivers, the Tumbler effectively acquires a total amount of $v_{tot} = \sum_{i \in [n]} v_{r_i}$.

Escrow phase. In this phase, each receiver shares the puzzle it obtained from the Tumbler with the corresponding sender. Subsequently, each sender creates an

escrow transaction that depends on the Tumbler's ability to solve a randomized version of this puzzle. For simplicity, assume n senders proceed in order (i.e., $\text{pk}_{s_1}, \text{pk}_{s_2}, \dots$). We emphasize that this ordering does not compromise unlinkability. β denotes the sum of $\{\beta_i\}_{i \in [n]}$, and let β be initially 0. Then, each sender pk_{s_i} (for $i = 1$ to n) interacts with the Tumbler as follows.

- $(\text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i}, \beta_i, \text{cpt}_\beta) \leftarrow \text{PuzzleSolveRq}(c_{r_i}, A_i, \hat{\sigma}_{r_i}, \text{cpt}_{\alpha_i}, \text{sk}_{s_i}, \text{pk}_t^\psi, \text{cmk}, \text{cpt}_\beta)$. The **PuzzleSolveRq** algorithm is run by the sender with sk_{s_i} . The sender provides evidence that the escrow transaction ctx_{esc} matches the same amount as some corresponding “promised” redeem transaction without harming unlinkability. Specifically:
 1. Use a random factor β_i to randomize the committed value c_{r_i} (denoted as c_{s_i}), puzzle A_i (denoted as A'_i), RSoRC signature $\hat{\sigma}_{r_i}$ (denoted as $\hat{\sigma}'_{s_i}$), and encrypted puzzle solution cpt_{α_i} (denoted as cpt'_{α_i}). This randomization breaks any direct link to the earlier puzzle promise phase while preserving the authorized binding $(c_{s_i}, A'_i, \hat{\sigma}'_{s_i})$.
 2. Create an incomplete confidential escrow transaction $\text{ctx}_{\text{esc}_i}^{\text{icp}} = (\text{pk}_{s_i}, \text{pk}_{tA}, c_{s_i}, \pi_{\text{legal}_i})$, where pk_{tA} is the public key of tumbler on Blockchain A.
 3. Generate a NIZK proof π_{valid_i} with public commitment key cmk and witness β_i . Intuitively, π_{valid_i} ensures that $\text{ctx}_{\text{esc}_i}^{\text{icp}}$ stems from one of the $\text{ctx}_{\text{red}}^{\text{icp}}$. In other words, they share the same amount.
 4. Presign $\text{ctx}_{\text{esc}_i}^{\text{icp}}$ with the sender's secret key sk_{s_i} and randomized puzzle A'_i , producing the presignature $\tilde{\sigma}_{s_i}$.
 5. Send $(\text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i})$ to the Tumbler. We denote $\text{HPKE}(\text{Enc}(\text{pk}_t^\psi, \beta))$ as cpt_β . Send $\text{cpt}_\beta = \text{cpt}_\beta \oplus_E \text{HPKE}(\text{Enc}(\text{pk}_t^\psi, \beta_i))$ ($\text{HPKE}(\text{Enc}(\text{pk}_t^\psi, \beta) \oplus_E \text{HPKE}(\text{Enc}(\text{pk}_t^\psi, \beta_i)) = \text{HPKE}(\text{Enc}(\text{pk}_t^\psi, \beta + \beta_i))$ holds) to $\text{pk}_{s_{i+1}}$ secretly, and if $i = n$, pk_n sends the last cpt_β to the Tumbler.
- $1/0 \leftarrow \text{ProcessVerify}(\{(\text{pk}_{s_i}, \text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i})\}_{i \in [n]}, \{c_{r_i}, t_i\}_{i \in [n]}, v_{\text{tot}}, \text{cpt}_\beta, \hat{\text{pk}}_t, \text{sk}_t^\psi, \text{state})$. The **ProcessVerify** algorithm is run by the Tumbler. The Tumbler runs this algorithm to decide whether to proceed to the next phase, i.e., by outputting 1 or 0. This algorithm outputs 1 when: 1) the Tumbler receiving n valid tuples from n registered senders, and 2) the last $\text{cpt}_\beta = \text{HPKE}(\text{Enc}(\text{pk}_t^\psi, \sum_{i \in [n]} \beta_i))$ can pass the Tumbler's check, within a pre-defined time bound. The passing of the above two checks ensures that the Tumbler can safely solve and accept all escrow transactions, and redeem them as a whole in the future, without learning the contents (e.g., the payment amount in each escrow). After verifying these tuples and the last β , the Tumbler proceeds to the next phase, or abort all requests otherwise.

Puzzle solving phase. During this phase, the Tumbler solves n received puzzles to finalize the corresponding escrow transactions. Once these escrows are settled, the solutions to the original n puzzles are revealed, allowing the associated redeem transactions to proceed. This process is formalized through two sub-routines: **ProcessE** (the Tumbler's puzzle-solving for each sender) and **ProcessR** (the redemption of coins by each receiver).

- $\{\text{ctx}_{\text{esc}_i}\}_{i \in [n]}/0 \leftarrow \text{ProcessEscrow}(\{(\text{ctx}_{\text{esc}_i}^{\text{icp}}, \tilde{\sigma}_{s_i}, \text{cpt}'_{\alpha_i})\}_{i \in [n]}, \text{sk}_t^\psi)$. The ProcessEscrow algorithm is run by the Tumbler. The Tumbler decrypts each cpt'_{α_i} using sk_t^ψ to obtain the solution α'_i and then get the escrow transaction onchain. That is, the Tumbler can convert each $\tilde{\sigma}_{s_i}$ into a valid signature σ_{s_i} , yielding a finalized $\text{ctx}_{\text{esc}_i}$. Here we emphasize that instead of explicit transmission, α'_i is visible to pk_{s_i} once $\text{ctx}_{\text{esc}_i}$ is paid to the Tumbler, thanks to the extractability property of the underlying adaptor signature method.
- $\text{ctx}_{\text{red}_i} \leftarrow \text{ProcessRedeem}(\text{ctx}_{\text{red}_i}^{\text{icp}}, \tilde{\sigma}_{tB_i}, \beta_i, \sigma_{s_i}, \tilde{\sigma}_{s_i}, A'_i)$. The ProcessVerify algorithm is mainly run by the receiver, after getting the puzzle solution from the sender. Specifically, once the sender gets σ_{s_i} from the $\text{ctx}_{\text{esc}_i}$, it extracts α'_i from σ_{s_i} and pre-signature $\tilde{\sigma}_{s_i}$, then the sender can recover α_i using the random factor β_i , which can be used by the receiver to complete the Tumbler's signature by forming the valid redeem transaction $\text{ctx}_{\text{red}_i}$.

4.2 Security Analysis

Theorem 1 *As designed in Section 3.4, $\text{LockMeld} = (\text{Setup}, \text{Register}, \text{PuzzleRequest}, \text{PuzzlePromise}, \text{PuzzleSolveRq}, \text{ProcessVerify}, \text{ProcessEscrow}, \text{ProcessRedeem})$, constitutes an atomic, unlinkable, double-spending-resistant, DoS-resistant, confidential, theft-resistant, and balanced cross-chain scheme for confidential blockchains.*

The formal proof of Theorem 1 is conducted within a game-based framework. Roughly, Theorem 1 can be decomposed into seven lemmas, each addressing a specific security property. These lemmas and their corresponding proofs are detailed in Appendix E.

5 Instantiation and Performance Analysis

5.1 Detailed Instantiation

In this section, we instantiate each algorithm in the last section.

- $\text{Setup}(1^\lambda)$. Run all underlying primitives' setup algorithm, such as NIZK. $\text{Setup}(1^\lambda)$, RSoRC.Setup(1^λ), etc., to output all public parameters.
- $\text{Register}(\text{sk}_t^*, \text{pk}_{r_i}, td_i)$. Each of the n senders runs this algorithm to finish the registration. We use the same register algorithm as in [18], to finish this phase. Then each sender sends its registration result to its corresponding receiver. That is, run the locking procedure in [18] to generate the registration identifier oid_i , the blind signature σ_i^* , and the sender keeps the unblind trapdoor td_i in secrete.
- $\text{PuzzleRequest}(\text{oid}_i, \sigma_i^*, td_i)$. In puzzle promise phase, this algorithm is run by each receiver with pk_{r_i} . The detailed algorithm is shown in Fig 10. The results are returned to the Tumbler with pk_{tB} .
- $\text{PuzzlePromise}(\text{oid}_i, \sigma'_i, v_r, t_r, \text{sk}_{tB}, \hat{\text{sk}}_t^\psi, \text{pk}_t^\psi, \text{pk}_t^*, \text{state})$. In the puzzle promise phase, the Tumbler runs this algorithm. Fig 11 shows the detailed procedure.

PuzzleRequest (oid _i , σ _r [*] , td _i)
1 : σ _r ← UnblindSig (td _i , σ _r [*]) 2 : σ' _r ← RandSig (σ _r) 3 : v _r ← $\mathcal{M}$, t _r ← $\mathcal{R}$ 4 : return (oid _i , σ' _r , v _r , t _r)

Fig. 10. Algorithm run by the Receiver in the *Puzzle promise phase*.

PuzzlePromise (oid _i , σ' _r , v _r , t _r , sk _{tB} , sk _t ^ψ , pk _t [*] , state)
// *Initially, state = ∅, v _{tot} = 0 * /
1 : If oid _i is registered ∨ SigVerify (pk _t [*] , σ' _r) = 0 ∨ pk _{tB} 's balance is less than v _r then abort 2 : c _r ← HCom.Com (v _r ; t _r) 3 : ctx _{red} ^{icp} ← CT.Create (sk _{tB} , pk _r , c _r) // *ctx _{red} ^{icp} = (pk _{tB} , pk _r , c _r , π _{legal}) * / 4 : (A _r , α _r) ← RandPuz.Gen (pp) 5 : cpt _{α_r} ← HPKE.Enc (pk _t ^ψ , α _r) 6 : pf ← NIZK (A _r , cpt _{α_r} ; α _r) // *Ensure cpt _{α_r} is solvable* / 7 : σ̃ _{tB} ← PreSig (sk _{tB} , ctx _{red} ^{icp} , A _r) 8 : σ̂ _r ← RSORC.RCSig (sk _t ^ψ , (A _r , c _r)) 9 : state ← state ∪ c _r // *state is also maintained by n cross-chain pairs * / 10 : v _{tot} ← v _{tot} + v _r 11 : return (ctx _{red} ^{icp} , A _r , σ̃ _{tB} , σ̂ _r , cpt _{α_r}) to pk _r ∧ return (state, v _{tot}) to pk _{tB} itself

Fig. 11. Algorithm run by the Tumbler in the *Puzzle promise phase*.

- **PuzzleSolveRq**(c_{r_i}, A_i, σ̂_{r_i}, cpt_{α_i}, sk_{s_i}, pk_t^ψ, cmk, cpt_β). In the escrow phase, this algorithm is run by each sender. The algorithm is shown in Fig 12. After the last sender running this algorithm, as described in Section 4.1, the last sender passes the final cpt_β = **HPKE.Enc**(pk_t^ψ, β) to the Tumbler.
- **ProcessVerify**({(pk_{s_i}, ctx_{esc_i}^{icp}, A'_i, σ̃_{s_i}, σ̂'_{s_i}, cpt'_{α_i}, π_{valid_i}) }_{i∈[n]}, {c_{r_i}, t_i}_{i∈[n]}, v_{tot}, cpt_β, pk_t, sk_t^ψ, state). In the escrow phase, this algorithm is run by the Tumbler, to determine whether to move on to the next phase. The detailed procedure is described in Fig. 13.
- **ProcessEscrow**({(ctx_{esc_i}^{icp}, σ̃_{s_i}, cpt'_{α_i}) }_{i∈[n]}, sk_t^ψ). The tumbler will run this algorithm for each senders if the verification passes and outputs 1 in the last step. The detailed algorithm is shown in Fig 14.
- **ProcessRedeem**(ctx_{red_i}^{icp}, σ̃_{tB_i}, β_i, σ_{s_i}, σ̃_{s_i}, A'_i). Fig 15 shows the receiver's redemption algorithm.

Here, ctx_{red}^{icp} is the incomplete form (i.e., without the signature σ_{tB}) of ctx_{red}. The latter, as described above, is a confidential transaction that locks coins

```

PuzzleSolveRq( $c_{r_i}, A_i, \hat{\sigma}_{r_i}, cpt_{\alpha_i}, sk_{s_i}, pk_t^\psi, cmk, cpt_\beta$ )

1:  $\beta_i \leftarrow \mathcal{R}$ 
2:  $(A'_i, c_{s_i}, \hat{\sigma}'_{s_i}) \leftarrow \text{RSoRC.RCRand}(A_i, c_{r_i}, \hat{\sigma}_{r_i}, \beta_i)$ 
3:  $cpt'_{\alpha_i} \leftarrow cpt_{\alpha_i} \oplus_E \text{HPKE.Enc}(pk_t^\psi, \beta_i)$ 
4:  $ctx_{esc_i}^{icp} \leftarrow \text{CT.Create}(sk_{s_i}, pk_{tA}, c_{s_i}) // *ctx_{esc_i}^{icp} = (pk_{s_i}, pk_{tA}, c_{s_i}, \pi_{\text{legal}})* /$ 
5:  $\tilde{\sigma}_{s_i} \leftarrow \text{PreSig}(sk_{s_i}, ctx_{esc_i}^{icp}, A'_i)$ 
6: for  $j = 1$  to  $n$  do  $commits \leftarrow commits \cup \{c_{r_j} \oplus_C c_{s_i}^{-1}\}$ 
   // *Initially,  $commits = \emptyset$ , and then turn into a standard OOOM commitments set */
   // *Let  $l$  be the index of the corresponding commitment in state */
   // *  $commits[l] = \text{HCom.Com}(0; \beta_i)$  */
7:  $\pi_{\text{valid}_i} \leftarrow \text{OOOM.Prove}(cmk, (commits, (l, \beta_i)))$ 
8:  $cpt_\beta \leftarrow cpt_\beta \oplus_E \text{HPKE.Enc}(pk_t^\psi, \beta_i) // *cpt_\beta = \text{HPKE.Enc}(pk_t^\psi, \beta + \beta_i) */$ 
9: return  $(ctx_{esc_i}^{icp}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, cpt'_{\alpha_i}, \pi_{\text{valid}_i}, \beta_i)$  to Tumbler,
   and return  $cpt_\beta$  to the next sender
   // *The last sender return  $cpt_{beta}$  to the Tumbler */

```

Fig. 12. Algorithm run by the Sender in the *Escrow phase*.

from Tumbler. Fig 16 provides ctx_{red} 's detailed construction. The rules of this transaction figure are the same as those in [2]. Roughly, once ctx_{esc} is on-chain, the transaction amount c_r it contains can be redeemed only by the owner of pk_r (with the signature σ_{tB} from Tumbler on Blockchain B). Otherwise, it will return to pk_{tB} if time exceeds the time limit Δ .

ctx_{esc} shown in Fig. 17 is a confidential transaction that locks coins from sender, and can be published in this phase. It has a construction and function similar to ctx_{red} except for the accounts engaged.

5.2 Performance Analysis

We provide a proof-of-concept implementation of LockMeld using the Truffle Suite [17]. To simulate a confidential blockchain, we deployed a Solidity smart contract on the Ethereum Virtual Machine (EVM) via Ganache [8]. The core protocol logic for the Receiver, Sender, and Tumbler is implemented in JavaScript. Benchmarking was conducted on a system equipped with a 2.6 GHz Intel Core i7 processor. The source code is available at <https://github.com/Inivia/LockMeld.git>. Table 3 presents the evaluation results. The performance of LockMeld across varying numbers of cross-chain user pairs (n) exhibits the following:

- *Time Cost.* As n increases from 8 to 64, each sender's time cost grows moderately (3.881 s to 10.423 s). In contrast, each receiver's time cost remains

```

ProcessVerify( $\{(\text{pk}_{s_i}, \text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i})\}_{i \in [n]}, \{c_{r_i}, t_i\}_{i \in [n]}, v_{\text{tot}}, \text{cpt}_{\beta}, \hat{\text{pk}}_t, \text{sk}_t^{\psi}, \text{state}$ )
1 : if  $\exists i \in [n]$  sat.  $\text{PreVf}(\text{pk}_{s_i}, \text{ctx}_{\text{esc}_i}^{\text{icp}}, \tilde{\sigma}_{s_i}, A'_i) = 0 \vee \text{RSoRC.RCVerify}(\hat{\text{pk}}_t, \hat{\sigma}'_{s_i}, A'_i, c_{s_i})$ 
    $= 0 \vee \text{OOM.Vf}(\text{cmk}, \text{state}, \pi_{\text{valid}_i}) = 0$  then return 0
2 : if  $\exists i \in [n]$  sat.  $\alpha'_i \leftarrow \text{HPKE.Dec}(\text{sk}_t^{\psi}, \text{cpt}'_{\alpha_i}) \wedge \alpha'_i$  is not the solution of  $A'_i$ 
   then return 0
3 :  $\beta \leftarrow \text{HPKE.Dec}(\text{sk}_t^{\psi}, \text{cpt}_{\beta})$ 
4 : if  $\text{HCom.Com}(0; \beta) \oplus_C \text{HCom.Com}(v_{\text{tot}}; 0) \oplus_C \text{HCom.Com}(0; \sum_{i \in [n]} t_i) = \oplus_C (c_{s_i})_{i \in [n]}$ 
5 : return 1

```

Fig. 13. Algorithm run by the Tumbler in the *Escrow phase*.

```

ProcessEscrow( $\{(\text{ctx}_{\text{esc}_i}^{\text{icp}}, \tilde{\sigma}_{s_i}, \text{cpt}'_{\alpha_i})\}_{i \in [n]}, \text{sk}_t^{\psi}$ )
1 :  $\alpha'_i \leftarrow \text{HPKE.Dec}(\text{sk}_t^{\psi}, \text{cpt}'_{\alpha_i})$ 
2 :  $\sigma_{s_i} \leftarrow \text{Adapt}(\tilde{\sigma}_{s_i}, \alpha'_i)$ 
3 : complete  $\text{ctx}_{\text{esc}_i}^{\text{icp}}$  to  $\text{ctx}_{\text{esc}_i} = \text{ctx}_{\text{esc}_i}^{\text{icp}} \cup \sigma_{s_i}$ 
   // *This publishes  $\sigma_{s_i}$  */
4 : return  $\{\text{ctx}_{\text{esc}_i}\}_{i \in [n]}/0$ 
   // *Tumbler can redeem  $v_{\text{tot}}$  with  $\beta + \sum_{i \in [n]} t_i$  *

```

Fig. 14. Algorithm run by the Tumbler in the *Puzzle solving phase*.

```

ProcessRedeem( $\text{ctx}_{\text{red}_i}^{\text{icp}}, \tilde{\sigma}_{tB_i}, \beta_i, \sigma_{s_i}, \tilde{\sigma}_{s_i}, A'_i$ )
1 :  $\alpha'_i \leftarrow \text{Ext}(\sigma_{s_i}, \tilde{\sigma}_{s_i}, A'_i)$  // *Run by the Sender */
2 :  $\alpha_i = \alpha'_i - \beta_i$ 
   // *Run by the sender, pass  $\alpha_i$  to the Reciever */
3 :  $\sigma_{t_i} \leftarrow \text{Adapt}(\tilde{\sigma}_{tB_i}, \alpha_i)$ 
4 : if  $\text{Vf}(\text{ctx}_{\text{red}_i}^{\text{icp}}, \sigma_{t_i}) = 0$  then return 0
5 : complete  $\text{ctx}_{\text{red}_i}^{\text{icp}}$  to  $\text{ctx}_{\text{red}_i} = \text{ctx}_{\text{red}_i}^{\text{icp}} \cup \sigma_{t_i}$ 
6 : return  $\text{ctx}_{\text{red}_i}$ 

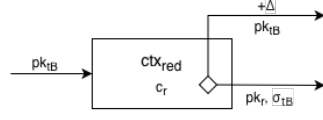
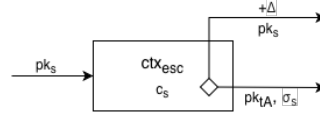
```

Fig. 15. Algorithm run by the Receiver in the *Puzzle solving phase*.

consistently low (about 0.01 s), while the tumbler's time cost escalates considerably (21.509 s to 255.510 s).

- *Communication and Gas Costs.* The total communication cost scales almost linearly with n , rising from 57 KB ($n=8$) to 519 KB ($n=64$). Similarly, on-chain gas costs on Chain A/B increase from 2295 K units to 5750 K units. Compared to the baseline (1863 K units), these increments reflect additional smart contract operations required by larger numbers of user pairs.

Overall, LockMeld provides moderate sender costs, low receiver overhead, and predictable communication/gas scaling. However, at larger scales, the tumbler's time cost emerges as the primary bottleneck. This overhead mainly attributes to the choice of HPKE in PuzzlePromise. Specifically, we use Castagnos-Laguillaumie encryption to encrypt α_i ($\alpha_i \leftarrow \mathbb{Z}_p$), which accounts for over 87% of the time. Note that although (Twisted) ElGamal scheme is more efficient, it is not qualified for our protocol due to its limited message space. In particular, the security of RandPuz would be compromised should the message space of α_i be small enough to enable brute-force recovery in the decryption of (Twisted) ElGamal. Further

**Fig. 16.** Construction of ctx_{red} .**Fig. 17.** Construction of ctx_{esc} .

parallelizing operations in the PuzzlePromise and PuzzleSolveRq phases could enhance Tumbler's performance.

Table 3. Cost Analysis of LockMeld Across All Parties

Number of Cross-Chain User Pairs	Each Sender's Time Cost (s)	Each Receiver's Time Cost (s)	Tumbler's Time Cost (s)	All Parties' Total Communication Cost (KB)	Chain A/B's Gas Cost (Units)
Baseline ¹	-	-	1.430	-	1863 K
$n=8$	3.881	0.012	21.509	57	2295 K
$n=16$	4.045	0.012	46.849	123	2789 K
$n=32$	5.499	0.012	105.388	255	3776 K
$n=64$	10.423	0.011	255.510	519	5750 K

¹ The baseline experiment measures the fundamental costs of smart contract deployment and the execution of independent transactions on each blockchain.

6 Conclusion

In this paper, we introduce LockMeld, the first privacy-preserving cross-chain protocol tailored for account-based confidential blockchains utilizing homomorphic commitments. LockMeld addresses two fundamental challenges in this field: achieving cross-chain unlinkability without compromising transaction availability, and supporting arbitrary payment amounts while maintaining confidentiality.

We formally prove the security and privacy properties of LockMeld through a rigorous game-based framework. Furthermore, our implementation demonstrates the practical feasibility of the protocol.

Acknowledgment. This work is supported by the National Science and Technology Major Project (Grant No. 2022ZD0120304) and Shanghai Science and Technology Innovation Action Plan (Grant No. 24BC3200300, 24BC3200500). Yu Long and Xian Xu are corresponding authors.

References

1. Augusto, A., Belchior, R., Correia, M., Vasconcelos, A., Zhang, L., Hardjono, T.: Sok: Security and privacy of blockchain interoperability. In: S&P (2024)
2. Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostakova, K., Maffei, M., Moreno-Sanchez, P., Riahi, S.: Generalized bitcoin-compatible channels. IACR Cryptol. ePrint Arch. **2020**, 476 (2020)
3. Bünz, B., Agrawal, S., Zamani, M., Boneh, D.: Zether: Towards Privacy in a Smart Contract World. In: FC. pp. 423–443. Springer (2020)
4. Campanelli, M., Hall-Andersen, M.: Veksel: Simple, efficient, anonymous payments with large anonymity sets from well-studied assumptions. In: AsiaCCS (2022)

5. Castagnos, G., Laguillaumie, F.: Linearly homomorphic encryption from. In: Cryptographers' Track at the RSA Conference. pp. 487–505. Springer (2015)
6. Chen, Y., Ma, X., Tang, C., Au, M.H.: PGC: Decentralized Confidential Payment System with Auditability. In: ESORICS (2020)
7. Du, J., Ge, Z., Long, Y., Liu, Z., Sun, S., Xu, X., Gu, D.: Mixct: Mixing confidential transactions from homomorphic commitment. In: European Symposium on Research in Computer Security. pp. 763–769. Springer (2022)
8. Ganache: (2022), <https://archive.trufflesuite.com/ganache/>
9. Glaeser, N., Maffei, M., Malavolta, G., Moreno-Sanchez, P., Tairi, E., Thyagarajan, S.A.K.: Foundations of coin mixing services. In: 2022 ACM CCS
10. Guo, Y., Xu, M., Cheng, X., Yu, D., Qiu, W., Qu, G., Wang, W., Song, M.: {zkCross}: A novel architecture for {Cross-Chain}{Privacy-Preserving} auditing. In: USENIX Security 24. pp. 6219–6235 (2024)
11. Han, P., Yan, Z., Yang, L.T., Bertino, E.: P2c2t: Preserving the privacy of cross-chain transfer. In: S&P. pp. 1474–1492. IEEE (2025)
12. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. Available at SSRN 3440802 (2008)
13. Platform, E.B.A.: Ethereum blockchain app platform, <https://www.ethereum.org/>
14. Qin, X., Pan, S., Mirzaei, A., Sui, Z., Ersoy, O., Sakzad, A., Esgin, M.F., Liu, J.K., Yu, J., Yuen, T.H.: Blindhub: Bitcoin-compatible privacy-preserving payment channel hubs supporting variable amounts. In: 2023 SP
15. Ruffing, T., Moreno-Sanchez, P.: ValueShuffle: Mixing Confidential Transactions for Comprehensive Transaction Privacy in Bitcoin. In: FC. pp. 133–154 (2017)
16. Sasson, E.B., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from bitcoin. In: 2014 IEEE symposium on security and privacy. pp. 459–474. IEEE (2014)
17. Suite, T.: (2022), <https://trufflesuite.com/>
18. Tairi, E., Moreno-Sanchez, P., Maffei, M.: A 2 l: Anonymous atomic locks for scalability in payment channel hubs. In: 2021 IEEE symposium on security and privacy (SP). pp. 1834–1851. IEEE (2021)
19. Wang, C., Ge, Z., Long, Y., Xu, X., Sun, S.F., Gu, D.: Blindshuffler: Universal and trustless mixing for confidential transactions. In: ACM AsiaCCS. pp. 1430–1446 (2024)

A Definition of Additive Homomorphism Message Hiding Schemes

A.1 Assumptions

The cryptography assumptions used in our work is described as follows.

Definition 8 (One Way Permutation) *We define OWP as a one-way permutation defined on $\mathcal{X}$ where $|\mathcal{X}| \geq 2^\lambda$. Then OWP is hard-to-invert, if for any PPT adversary $\mathcal{A}$, $\Pr[x \leftarrow \mathcal{X}, y = \text{OWP}(x) | \mathcal{A}(\text{OWP}, y) = x] \leq \text{negl}(\lambda)$.*

Definition 9 (Discrete Logarithm Assumption) *The discrete logarithm (DL) assumption holds, if for any PPT adversary $\mathcal{A}$, given $(\mathbb{G}, p, g) \leftarrow \text{GroupGen}(1^\lambda)$ and a random element $h \in \mathbb{G}$, $\Pr[x \leftarrow \mathbb{Z}_p, h = g^x | \mathcal{A}(\mathbb{G}, p, g, h) = x] = \text{negl}(\lambda)$.*

A.2 Additive Homomorphism Commitment

An additive homomorphism commitment scheme HCom consists of the following algorithms. Let $\mathcal{M}, \mathcal{R}, \mathcal{C}$ denote the space of the message, the randomness, and the commitment, respectively.

- $\text{pp} \leftarrow \text{HCom.Gen}(1^\lambda)$. This algorithm takes as input the security parameter 1^λ and generates the public parameter pp which will be used implicitly in the following algorithm.
- $c \leftarrow \text{HCom.Com}(m; r)$. On input $m \in \mathcal{M}$ and a randomly chosen hiding factor $r \leftarrow \mathcal{R}$, this algorithm outputs the commitment result c .

Beyond the standard hiding and binding properties that all common commitment schemes promise, an additive homomorphic commitment scheme also satisfies the property described below.

Definition 10 (Additive Homomorphism) *A commitment scheme HCom is additive homomorphism, if for all $\text{pp} \leftarrow \text{HCom.Gen}(1^\lambda)$, and $\forall m_1, m_2 \in \mathcal{M}$ and $\forall r_1, r_2 \in \mathcal{R}$, the following equation holds:*

$$\text{HCom.Com}(m_1; r_1) \oplus_C \text{HCom.Com}(m_2; r_2) = \text{HCom.Com}(m_1 \oplus_M m_2; r_1 \oplus_R r_2)$$

where $\oplus_M, \oplus_R, \oplus_C$ denotes the group operations in $\mathcal{M}, \mathcal{R}$, and $\mathcal{C}$, respectively

One of the most famous instantiation of HCom is the **Pedersen Commitment**[7].

A.3 Additively Homomorphic Public Key Encryption

An additively homomorphic PKE scheme HPKE consists of the following algorithms. Let $\mathcal{M}, \mathcal{KPT}$ denote the message space, the key space, and the ciphertext space.

- $\text{pp} \leftarrow \text{HPKE.Setup}(1^\lambda)$. On input the security parameter 1^λ , the setup algorithm generates the public parameter pp , which will be used implicitly in the following algorithm.
- $(\text{pk}^\psi, \text{sk}^\psi) \leftarrow \text{HPKE.Gen}(\text{pp})$. On input the public parameter pp , this algorithm outputs a key pair (pk, sk) .
- $\text{cpt} \leftarrow \text{HPKE.Enc}(\text{pk}^\psi, m)$. On input a public key pk^ψ and a message $m \in \mathcal{M}$, this algorithm outputs a ciphertext $\text{cpt} \in \mathcal{CPT}$.
- $m \leftarrow \text{HPKE.Dec}(\text{sk}^\psi, \text{cpt})$. Given a secret key sk^ψ and a ciphertext $\text{cpt} \in \mathcal{CPT}$, this algorithm retrieves and outputs a message $m \in \mathcal{M}$.

Here we consider the secure HPKE schemes which have IND-CPA security and exhibit additive homomorphism aligns with the corresponding property defined above for the HCom scheme. the following equation holds:

$$\text{HPKE.Enc}(\text{pk}^\psi, m_1) \oplus_E \text{HPKE.Enc}(\text{pk}^\psi, m_2) = \text{HPKE.Enc}(\text{pk}^\psi, m_1 \oplus_M m_2)$$

where $\oplus_E$ and $\oplus_M$ denotes the group operations in $\mathcal{CPT}$ and $\mathcal{M}$ respectively. Notable instantiations of such a scheme are the (Twisted) ElGamal encryption scheme [6] and the CL (Castagnos-Laguillaumie) encryption scheme[5]. In our protocol instantiation, we use CL encryption as HPKE scheme for it has the same message space as the solution in RandPuz.

B Definition of NIZK

We use NIZK scheme to prove that a public statement x belongs to an NP language $\mathcal{L}$ with a witness w without revealing w , where $(x; w) \in R_{\mathcal{L}}$. An NIZK scheme consists of following algorithms.

- $\text{pp} \leftarrow \text{NIZK.Setup}(1^\lambda)$. On input security parameter, output the public parameters pp used as implicit input of the following two algorithms.
- $\pi \leftarrow \text{NIZK.Prove}(x, w)$. On input the statement x and the witness w , output a proof π .
- $0/1 \leftarrow \text{NIZK.Vf}(x, \pi)$. On input a statement x and a proof π , output 0 if reject and 1 otherwise.

We require the completeness, knowledge-soundness, and zero-knowledge properties of NIZK. *Completeness* means the prover who knows a witness of a statement could always convince the verifier to accept. *Knowledge-soundness* holds if the prover “knows” a witness for the accepted statement. *Zero-knowledge* property indicates that the verifier cannot get additional information except that the statement is true.

C Definition of Adaptor Signature

The properties of pre-signature adaptability, witness extractability and aEUF-CMA security of an adaptor signature scheme are formally defined as follows. Let $\mathcal{M}$ and $\mathcal{S}$ denote the message space and the pre-signature space.

Definition 11 (Pre-Signature Adaptability) *An adaptor signature scheme is pre-signature adaptable if, for any $\lambda \in \mathbb{N}$, any message $m \in \mathcal{M}$, any state-ment/witness pair $(Y, y) \in R$, any pre-signature $\tilde{\sigma} \in \mathcal{S}$ that satisfies*

$$\text{PrVf}(\text{pk}, m, \tilde{\sigma}, Y), \text{ it holds that } \text{Pr}[\text{Vf}(\text{pk}, m, \text{Adapt}(\tilde{\sigma}, y)) = 1] = 1.$$

Definition 12 (Witness Extractability) *An adaptor signature scheme is witness extractable if, for any PPT adversary $\mathcal{A}$, it holds that $\text{Adv}_{\mathcal{A}, \text{AdaptorSig}}^{\text{extract}}(\lambda) = \text{Pr}[\text{aWitExt}_{\text{AdaptorSig}}^{\mathcal{A}} = 1] \leq \text{negl}(\lambda)$, where $\text{aWitExt}_{\text{AdaptorSig}}^{\mathcal{A}}$ is defined as follows.*

$\text{aWitExt}_{\text{AdaptorSig}}^{\mathcal{A}}(1^\lambda)$	$\mathcal{O}_S(m)$
1 : $\mathcal{Q} := \emptyset$	1 : $\sigma \leftarrow \text{Sig}(\text{sk}, m)$
2 : $(\text{sk}, \text{pk}) \leftarrow \text{KGen}(1^\lambda)$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
3 : $(m, Y) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{pS}(\cdot, \cdot)}(\text{pk})$	3 : return σ
4 : $\tilde{\sigma} \leftarrow \text{PreSig}(\text{sk}, m, Y)$	$\mathcal{O}_{pS}(m, Y)$
5 : $\sigma \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{pS}(\cdot, \cdot)}(\tilde{\sigma})$	1 : $\tilde{\sigma} \leftarrow \text{PreSig}(\text{sk}, m, Y)$
6 : $y' := (\sigma, \tilde{\sigma}, Y)$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
7 : return $m \notin \mathcal{Q} \wedge (Y, y') \notin R \wedge \text{Vf}(\text{pk}, m, \sigma)$	3 : return $\tilde{\sigma}$

Definition 13 (aEUF-CMA Security) An adaptor signature scheme is aEUF-CMA secure if, for any PPT adversary $\mathcal{A}$, it holds that $\text{Adv}_{\mathcal{A}, \text{AdaptorSig}}^{\text{aeuf-cma}}(\lambda) = \Pr[\text{aSigForge}_{\text{AdaptorSig}}^{\mathcal{A}} = 1] \leq \text{negl}(\lambda)$, where $\text{aSigForge}_{\text{AdaptorSig}}^{\mathcal{A}}$ is defined as follows.

$\text{aSigForge}_{\text{AdaptorSig}}^{\mathcal{A}}(1^\lambda)$	$\mathcal{O}_S(m)$
1 : $\mathcal{Q} := \emptyset$	1 : $\sigma \leftarrow \text{Sig}(\text{sk}, m)$
2 : $(\text{sk}, \text{pk}) \leftarrow \text{KGen}(1^\lambda)$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
3 : $(m) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{pS}(\cdot, \cdot)}(\text{pk})$	3 : return σ
4 : $(Y, y) \leftarrow \text{RGen}(1^\lambda)$	$\mathcal{O}_{pS}(m, Y)$
5 : $\tilde{\sigma} \leftarrow \text{PreSig}(\text{sk}, m, Y)$	1 : $\tilde{\sigma} \leftarrow \text{PreSig}(\text{sk}, m, Y)$
6 : $\sigma \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{pS}(\cdot, \cdot)}(\tilde{\sigma})$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
7 : $y' := (\sigma, \tilde{\sigma}, Y)$	3 : return $\tilde{\sigma}$
8 : return $m \notin \mathcal{Q} \wedge \text{Vf}(\text{pk}, m, \sigma)$	

D Definition of Oracles

- $\text{sk} \leftarrow \mathcal{O}_{\text{crt}}(\text{pk})$. This oracle describes the ability of adversary $\mathcal{A}$ to corrupt honest users and obtain their secret keys. When called, challenger $\mathcal{C}$ searches

Table 4. Structure of L_{pp} and L_{ps} used in the security model

$L_{pp} :$	pk_r	Corrupt	ctx_{red}	v_r	A		
$L_{ps} :$	pk_s	Corrupt	ctx_{esc}	v_s	A'	pk_r	A

the system for the secret key corresponding to $\mathbf{pk}$ and gives it to the adversary $\mathcal{A}$. If $\mathbf{pk}$ exists in L_{pp} or L_{ps} , $\mathcal{C}$ sets the corresponding **Corrupt** field to 1 in that table.

- $\mathcal{O}_{\text{LockMeld}}((\text{oid}, \sigma_r^*, td, v_r, t_r, \mathbf{pk}_{tB}, \mathbf{pk}_r) / (c_{r_i}, A_i, \hat{\sigma}_{r_i}, \text{cpt}_{\alpha_i}, \mathbf{sk}_{s_i}, \mathbf{pk}_t^\psi, \text{cmk}, \text{cpt}_\beta))$. The adversary $\mathcal{A}$ may instruct corrupted users (with keys $\mathbf{pk}_r$ or $\mathbf{pk}_s$) to make either puzzle promise requests or puzzle solving requests. The challenger $\mathcal{C}$ responds as follows:

- If the message $(\text{oid}, \sigma_r^*, td, v_r, t_r, \mathbf{pk}_{tB}, \mathbf{pk}_r)$ is received, $\mathcal{C}$ runs

$$(\text{ctx}_{\text{red}}^{\text{icp}}, A_r, \tilde{\sigma}_{tB}, \hat{\sigma}_r, \text{cpt}_{\alpha_r}, \text{state}, v_{\text{tot}}, \delta) \leftarrow \text{PuzzlePromise}(\text{oid}, \sigma_r', v_r, t_r, \mathbf{sk}_{tB}, \hat{\mathbf{sk}}_t, \mathbf{pk}_t^\psi, \mathbf{pk}_r, \text{state}),$$

and adds $(\mathbf{pk}_r, 1, \text{ctx}_{\text{red}}^{\text{icp}}, v_r, A)$ to the list L_{pp} if $\delta = 1$.

- If the message $(c_{r_i}, A_i, \hat{\sigma}_{r_i}, \text{cpt}_{\alpha_i}, \mathbf{sk}_{s_i}, \mathbf{pk}_t^\psi, \text{cmk}, \text{cpt}_\beta)$ is received, $\mathcal{C}$ runs

$$\delta_1 \leftarrow \text{ProcessVerify}(\{(\mathbf{pk}_{s_i}, \text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i})\}_{i \in [n]}, \{c_{r_i}, t_i\}_{i \in [n]}, v_{\text{tot}}, \text{cpt}_\beta, \hat{\mathbf{sk}}_t, \mathbf{sk}_t^\psi, \text{state}),$$

If $\delta_1 = 0$, aborts. Otherwise runs

$$\delta_2 \leftarrow \text{ProcessEscrow}(\{(\text{ctx}_{\text{esc}_i}^{\text{icp}}, \tilde{\sigma}_{s_i}, \text{cpt}'_{\alpha_i})\}_{i \in [n]}, \mathbf{sk}_t^\psi),$$

and adds $(\mathbf{pk}_s, 1, \text{ctx}_{\text{esc}}^{\text{icp}}, \perp, A', \mathbf{pk}_r, \perp)$ to the list L_r if $\delta_2 = 1$.

- $(\text{ctx}_{\text{red}}^{\text{icp}}, A_r, \tilde{\sigma}_{tB}, \hat{\sigma}_r, \text{cpt}_{\alpha_r}, \text{state}) \leftarrow \mathcal{O}_{\text{PP}}(v_r, \mathbf{pk}_{tB}, \mathbf{pk}_r)$
Adversary $\mathcal{A}$ queries this oracle to induce an honest user $\mathbf{pk}_r$ to create a puzzle promise request for value v . The challenger $\mathcal{C}$ runs:

$$(\text{oid}, \sigma_r', v_r, t_r) \leftarrow \text{PuzzleRequest}(\text{oid}, \sigma_r^*, td, v_r, t_r, \mathbf{pk}_{tB}, \mathbf{pk}_r),$$

$$(\text{ctx}_{\text{red}}^{\text{icp}}, A_r, \tilde{\sigma}_{tB}, \hat{\sigma}_r, \text{cpt}_{\alpha_r}, \text{state}, v_{\text{tot}}, \delta) \leftarrow \text{PuzzlePromise}(\text{oid}, \sigma_r', v_r, t_r, \mathbf{sk}_{tB}, \hat{\mathbf{sk}}_t, \mathbf{pk}_t^\psi, \mathbf{pk}_r, \text{state}).$$

Then add $(\mathbf{pk}_r, 0, \text{ctx}_{\text{red}}, v_r, A_r)$ to L_{pp} if $\delta = 1$.

- $(\text{ctx}_{\text{esc}_i}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \pi_{\text{valid}_i}) \leftarrow \mathcal{O}_{\text{PS}}(\mathbf{pk}_{s_i})$ Adversary $\mathcal{A}$ queries this oracle to induce honest user $\mathbf{pk}_{s_i}$ to create a puzzle solving request. The honest user receives information from a puzzle promise outputs, then $\mathcal{C}$ runs:

$$(\text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i}, \beta_i, \text{cpt}_\beta) \leftarrow \text{PuzzleSolveRq}(c_{r_i}, A_i, \hat{\sigma}_{r_i}, \text{cpt}_{\alpha_i}, \mathbf{sk}_{s_i}, \mathbf{pk}_t^\psi, \text{cmk}, \text{cpt}_\beta),$$

after receiving all Puzzle solving requests in the epoch, $\mathcal{C}$ runs

$$\delta_1 \leftarrow \text{ProcessVerify}(\{(\mathbf{pk}_{s_i}, \text{ctx}_{\text{esc}_i}^{\text{icp}}, A'_i, \tilde{\sigma}_{s_i}, \hat{\sigma}'_{s_i}, \text{cpt}'_{\alpha_i}, \pi_{\text{valid}_i})\}_{i \in [n]}, \{c_{r_i}, t_i\}_{i \in [n]},$$

$$v_{\text{tot}}, \text{cpt}_{\beta}, \hat{\text{pk}}_t, \text{sk}_t^{\psi}, \text{state}),$$

and if $\delta_1 = 0$, aborts. Otherwise runs

$$\delta_2 \leftarrow \text{ProcessEscrow}(\{(\text{ctx}_{\text{esc}_i}^{\text{icp}}, \tilde{\sigma}_{s_i}, \text{cpt}'_{\alpha_i})\}_{i \in [n]}, \text{sk}_t^{\psi}),$$

Then add $(\text{pk}_{s_i}, 0, \text{ctx}_{\text{esc}_i}^{\text{icp}}, v_{s_i}, A'_i, \text{pk}_{r_i}, A_i)$ to L_{ps} if $\delta_2 = 1$.

For ease of illustration, we will collectively refer to the set of aforementioned oracles as OCT.

E Security Proof for LockMeld Scheme

For ease of description, we define event Break as follows: the adversary obtains the solution to the puzzle directly from the puzzle or ciphertext without any additional information. When Break occurs, the Security of Radomizable Puzzle or the CPA security of Homomorphic PKE is broken. So we have:

$$\Pr[\text{Break occurs}] \leq \text{Adv}_{\mathcal{A}, \text{RP}}^{\text{security}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{cpa}}(\lambda).$$

The following proofs proceed under the assumption that Break does not occur, and we will add it to the conclusion.

E.1 Lemmas

Lemma 1 *Under the assumptions of aEUF-CMA security, pre-signature adaptability, and witness extractability of the underlying adaptor signature scheme, the scheme LockMeld is atomic.*

Lemma 2 *Under the zero-knowledge properties of the OOOM proof system, the hiding property of commitments, and the class-hiding property of RSoRC, LockMeld is unlinkable.*

Lemma 3 *Leveraging the aEUF-CMA security of the adaptor signature scheme, the CPA security of the public-key encryption (PKE) scheme, and the security of the randomizable puzzle mechanism, LockMeld prevents double spending.*

Lemma 4 *Under the EUF-CMA security of blinded randomizable signatures, LockMeld is DoS resistant.*

Lemma 5 *Based on the zero-knowledge properties of the OOOM proof system, the hiding property of commitments, and the class-hiding property of RSoRC, LockMeld provides confidentiality.*

Lemma 6 *By relying on the aEUF-CMA security of adaptor signatures, the CPA security of PKE, and the security of randomizable puzzles, LockMeld prevents theft.*

Lemma 7 *Using the soundness of the OOOM proof system, the aEUF-CMA security of adaptor signatures, the EUF-CMA security of RSoRC, the CPA security of PKE, and the security of randomizable puzzles, LockMeld is balanced.*

E.2 Proof of Lemma 1

We notice that if $\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{atomicity}}(\lambda)$ is non-negligible, we can construct an algorithm that break the witness extractability property of adapt signature scheme. Thus,

$$\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{atomicity}}(\lambda) = \Pr[\text{Atomicity}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \leq \text{Adv}_{\mathcal{A}, \text{AdaptorSig}}^{\text{extract}}(\lambda).$$

E.3 Proof of Lemma 2

We use a sequence of games to prove that $\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{unlink}}(\lambda)$ is negligible. The specific structure of games is shown as follows.

Game 0 is the real experiment $\text{Unlinkability}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda)$ described in Definition 2.

Game 1 is basically the same as Game 0, except that the challenger $\mathcal{C}$ uses the simulator OOOM.Sim instead of prover OOOM.Prove to calculate $\pi_{\text{valid}}^{(b)}$. We consider that Game 1 is computationally indistinguishable from Game 0. If this statement is not true, we can define an algorithm $\mathcal{D}_1$ to break the zero-knowledge property of OMMM.

$\mathcal{D}_1$ proceeds as follows. $\mathcal{D}_1$ sends $(\text{cmk}, \text{state}, (l, \beta))$ to his own challenger and receives π from it. Challenger computes $\pi \leftarrow \text{Prove}(\text{cmk}, \text{state}, (l, \beta))$ when $\beta = 0$, and $\pi \leftarrow \text{Sim}(\text{cmk}, \text{state})$ when $\beta = 1$. It is obvious that $\mathcal{D}_1$ simulates Game 0 when $\beta = 0$, and simulates Game 1 when $\beta = 1$. After the adversary distinguishing Game 0 and Game 1 outputs β' , $\mathcal{D}_1$ can send β' to his own challenger. Thus,

$$\left| \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game1}}(\lambda) - \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game0}}(\lambda) \right| \leq \text{Adv}_{\mathcal{A}, \text{OOOM}}^{\text{zk}}(\lambda).$$

Game 2 is the same as Game 1, except that the RSoRC tuple $(A', c_s, \hat{\sigma}')$ is replaced with a tuple (A^*, c^*, σ^*) randomly sampled in the whole space. We claim that Game 2 is computationally indistinguishable from Game 1 by defining an algorithm $\mathcal{D}_2$ that can break the class-hiding property of RSoRC.

$\mathcal{D}_2$ proceeds as follows. When the adversary distinguishing Game 1 and Game 2 send $(\text{ctx}_{\text{red}})$, $\mathcal{D}_2$ send it to its own challenger which outputs $(A', c_s, \hat{\sigma}')$ while $\beta = 0$ and (A^*, c^*, σ^*) while $\beta = 1$. We observe that $\mathcal{D}_2$ simulates Game 1 when $\beta = 0$, and Game 2 when $\beta = 1$, and it can pass β' output from the adversary to its challenger. Thus,

$$\left| \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game2}}(\lambda) - \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game1}}(\lambda) \right| \leq \text{Adv}_{\mathcal{A}, \text{RSoRC}}^{\text{chiding}}(\lambda).$$

We can notice that tuple $(A^*, c^*, \sigma^*, \pi)$ in Game 2 is completely have no connection with adversary $\mathcal{A}$, thus,

$$\left| \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game2}}(\lambda) \right| = 0.$$

In summary,

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{unlink}}(\lambda) &= \left| \Pr[\text{Unlinkability}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2} \right| \\ &\leq \text{Adv}_{\mathcal{A}, \text{OOM}}^{\text{zk}}(\lambda) + Q \cdot \text{Adv}_{\mathcal{A}, \text{RSRC}}^{\text{chiding}}(\lambda). \end{aligned}$$

E.4 Proof of Lemma 3

We suppose that the the probability of tumbler generating duplicated puzzles is negligible. Thus if there is a PPTadversary $\mathcal{A}$ can win the double-spent prevention game described in Definition 3 when Break does not occur, we can define an algorithm $\mathcal{D}_3$ to break the aEUF-CMA property of adaptor signature.

The algorithm $\mathcal{D}_3$ proceeds as follows.

$\mathcal{D}_3$ receives two puzzle request from adversary, then query its own challenger for two pre-signature. When receive one valid puzzle solver from adversary, query challenger for the complete sign σ . When adversary wins $\text{DSPrevention}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda)$ game, $\mathcal{D}_3$ randomly chose one of the signature of redeem transaction to challenger.

$$\Pr[a\text{EUF} - \text{CMA}] = \frac{1}{2} \Pr[\text{DSPrevention}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1 | \text{Not Break}]$$

Thus,

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{doublespend}}(\lambda) &= \Pr[\text{DSPrevention}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \\ &\leq 2\text{Adv}_{\mathcal{A}, \text{AdaptorSig}}^{\text{aeuf-cma}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{RP}}^{\text{security}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{cpa}}(\lambda) \end{aligned}$$

E.5 Proof of Lemma 4

The property of DoS-resistance is guaranteed by the Register phase, as stated in [18].

E.6 Proof of Lemma 5

We use a sequence of games to prove that $\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{confidentiality}}(\lambda)$ is negligible. The specific structure of games is shown as follows.

Game 0 is the real game experiment $\text{Confidentiality}_{\text{LockMeld}}^{\mathcal{A}}$ described in Definition 5.

Game 1 is the same as Game 0, except that the challenger $\mathcal{C}$ uses the simulator OOM.Sim instead of prover OOM.Prove to calculate $\pi_{\text{valid}}^{(b)}$. We consider that Game 1 is computationally indistinguishable from Game 0. If this statement is not true, we can define an algorithm $\mathcal{D}_4$ to break the zero-knowledge property of OMMM. The construction of $\mathcal{D}_4$ is similar to $\mathcal{D}_1$. Thus,

$$\left| \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game1}}(\lambda) - \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game0}}(\lambda) \right| \leq \text{Adv}_{\mathcal{A}, \text{OOM}}^{\text{zk}}(\lambda).$$

Game 2 is the same as Game 1, except that the challenger C use a random v^* instead of v_b in redeem transaction. Then the output of C is $(\text{ctx}_{\text{red}}^*, \text{ctx}_{\text{esc}}^{(b)}, \pi_{\text{valid}}^{(b)}, \hat{\sigma}^{(b)})$. We prove that Game 2 is computationally indistinguishable from Game 1 by design an algorithm $\mathcal{D}_5$, which can break the hiding property of commitment scheme.

$\mathcal{D}_5$ queries c^* from its own challenger instead of computing or sampling itself. $c^* = \text{Com}(v_b, r)$ when $\beta = 0$, and $c^* = \text{Com}(v^*, r)$ otherwise. It is clear that $\mathcal{D}_5$ simulates Game 1 and Game 2 respectively when $\beta = 0$ and $\beta = 1$. After the adversary distinguishing Game 1 and Game 2 outputs β' , $\mathcal{D}_5$ can send β' to his own challenger. Thus,

$$\left| \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game2}}(\lambda) - \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game1}}(\lambda) \right| \leq \text{Adv}_{\mathcal{A}, \text{C}}^{\text{hiding}}(\lambda).$$

Game 3 is the same as Game 2, except that the RSoRC tuple $(c_s, \hat{\sigma}')$ is replaced with a tuple (c^*, σ^*) randomly sampled in the whole space. Then the output of C is $(\text{ctx}_{\text{red}}^*, \text{ctx}_{\text{esc}}^*, \pi_{\text{valid}}^*, \sigma^*)$. We claim that Game 3 is computationally indistinguishable from Game 2 by defining an algorithm $\mathcal{D}_6$ that can break the class-hiding property of RSoRC. The construction of $\mathcal{D}_6$ is similar to $\mathcal{D}_2$. Thus,

$$\left| \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game3}}(\lambda) - \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game2}}(\lambda) \right| \leq \text{Adv}_{\mathcal{A}, \text{RSoRC}}^{\text{chiding}}(\lambda).$$

We can notice that tuple $(\text{ctx}_{\text{red}}^*, \text{ctx}_{\text{esc}}^*, \pi_{\text{valid}}^*, \sigma^*)$ in Game 3 is completely have no connection with the input of adversary $\mathcal{A}$. Thus,

$$\left| \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{game3}}(\lambda) \right| = 0.$$

In summary,

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{confidentiality}}(\lambda) &= \left| \Pr[\text{Confidentiality}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2} \right| \\ &\leq \text{Adv}_{\mathcal{A}, \text{OOM}}^{\text{zk}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{C}}^{\text{hiding}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{RSoRC}}^{\text{chiding}}(\lambda). \end{aligned}$$

E.7 Proof of Lemma 6

As described in Definition 6, we defined property balance in two different conditions.

Case 1: user theft. Define event $\mathcal{E}_{\text{uth}}$ to describe that $\mathcal{A}$ wins theft prevention game by successfully redeeming from an honest user. And this is the same case to Break. Thus,

$$\Pr[\mathcal{E}_{\text{uth}} \text{ occurs}] = 0.$$

Case 2: system theft. Define event $\mathcal{E}_{\text{sth}}$ to describe that $\mathcal{A}$ wins theft prevention game by successfully redeeming when there is no valid escrow. We derive that the possibility of event $\mathcal{E}_{\text{sth}}$ occurring is negligible by providing an algorithm $\mathcal{D}_8$, which can break the aEUF-CMA property of adaptor signature. The construction of $\mathcal{D}_8$ is similar to $\mathcal{D}_3$. Thus,

$$\Pr[\mathcal{E}_{\text{sth}} \text{ occurs}] \leq \text{Adv}_{\mathcal{A}, \text{AdaptorSig}}^{\text{aeuf-cma}}(\lambda).$$

In summary of above discussion,

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{doublespend}}(\lambda) &= \Pr[\text{TheftPrevention}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \\ &\leq \text{Adv}_{\mathcal{A}, \text{AdaptorSig}}^{\text{euf-cma}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{RP}}^{\text{security}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{cpa}}(\lambda) \end{aligned}$$

E.8 Proof of Lemma 7

As described in Definition 7, we defined property balance in two different condition.

Define event $\mathcal{E}_{\text{dpuz}}$ to describe the condition that $\mathcal{A}$ wins the Balance game in the following way. $\mathcal{A}$ provides a valid challenge $(\hat{\mathbf{p}}\mathbf{k}_t, \hat{\sigma}'_{s\mathcal{A}}, A'_{\mathcal{A}}, c_{s\mathcal{A}})$, but $L_{pp}[A'_{\mathcal{A}}].A \neq A_{\mathcal{A}}$, which means the randomized puzzle provided by $\mathcal{A}$ is not randomized from the one challenger generated in puzzle promise phase. That is to say, $\mathcal{A}$ can use this to get the solution of other puzzle and then sign the corresponding redeem transaction.

We prove that the possibility of event $\mathcal{E}_{\text{dpuz}}$ occurring is negligible by defining an algorithm $\mathcal{D}_9$ to break the EUF-CMA property of RSoRC. $\mathcal{D}_9$ simulates $\text{Balance}_{\text{LockMeld}}^{\mathcal{A}}$ by quering challenger instead of generate RSoRS signature Q times in challenge phase. Then $\mathcal{D}_9$ receive Q valid signature in Puzzle Solver from adversary. At least one of them is forged and uses different puzzle when $\mathcal{E}_{\text{dpuz}}$ happens. $\mathcal{D}_9$ randomly choose one and submit it to challenger. Then,

$$\text{Adv}_{\mathcal{A}, \text{RSoRC}}^{\text{euf-cma}}(\lambda) \geq \frac{1}{Q} \Pr[\mathcal{E}_{\text{dpuz}} \text{ occurs}]$$

Thus,

$$\Pr[\mathcal{E}_{\text{dpuz}} \text{ occurs}] \leq Q \cdot \text{Adv}_{\mathcal{A}, \text{RSoRC}}^{\text{euf-cma}}(\lambda).$$

In summary of above discussion,

$$\text{Adv}_{\mathcal{A}, \text{LockMeld}}^{\text{doublespend}}(\lambda) = \Pr[\text{Balance}_{\text{LockMeld}}^{\mathcal{A}}(1^\lambda) = 1] \leq Q \cdot \text{Adv}_{\mathcal{A}, \text{RSoRC}}^{\text{euf-cma}}(\lambda)$$